

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\agent-a07e44ec9585fcad0.jsonl`  
- SHA-256: `3bf49c40604227bc41be136f1d73e62e938ce99856f664574622f5e685f58c8a`  
- Source modified: `2026-06-22T03:21:00+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `subagents`  
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T03:18:35.027Z)

Map the khelsutra web SPA so I can build a "P2c first-run wizard" (3 steps: Gemini key / Compute / Videos) rendered from the engine's GET /api/capabilities. Work in C:\Users\avidu\Projects\khelsutra-p2c\web (this is avidu/khelsutra origin/master). Read-only; return a TIGHT structured map with exact file:line citations and short code snippets ? enough that I can write the wizard without re-reading everything.

Answer:

- **App mount + screen state.**** Read web/src/App.tsx. How does it decide which screen to show (e.g. IngestPanel when no videoId, the rally grid otherwise)? Quote the state + the conditional render. Where would a first-run <SetupWizard/> hook in, and what existing state (videoId, etc.) signals "first run / nothing done yet"?
- **API client pattern.**** Read web/src/api/client.ts + web/src/api/types.ts. Show the BASE/url pattern (same-origin '/api'), how an endpoint is added (e.g. processVideo/getJob), and the error handling (parseDetail/classifyError in lib/errors). I will add a getCapabilities() call ? show the exact idiom to mirror. Does anything already call /api/capabilities or /api/config?
- **/api/capabilities shape.**** The engine returns (confirmed): {api_version, engine_version, segmenters:[...], default_segmenter, ffmpeg:{ffmpeg:bool, ffprobe:bool, ffmpeg_path, ffprobe_path}, gpu:{nvidia_smi_present:bool}, gemini_key_present:bool, wasb_weights_present:bool, deployment:{profile, compute_target}}. Note which of these the wizard's 3 steps would use.
- **i18n catalogs + the parity guard.**** Where do the locale catalogs live (en/kn/hi/es/da/id)? Read the en catalog file + name the others. CRITICALLY: find the key-parity / placeholder-parity test/guard (the UX track added one) ? quote the file + what it enforces (so I know I MUST add new wizard.* keys to ALL 6 catalogs with matching keys/placeholders or CI fails). How are translations structured (nested namespaces? flat?) and how is t() used in a component (show one example, incl. interpolation)?
- **Component + test conventions.**** Show how an existing screen-level component is built + tested ? use IngestPanel.tsx + IngestPanel.test.tsx as the template (imports, t() usage, the test runner = vitest? testing-library? how it mocks fetch/api). Name the test setup file (web/src/test/setup.ts) and what it provides. What's the exact `npm` script to run tests, typecheck, and build (from web/package.json)?
- **Persistence for "first-run done".**** Is there any localStorage usage already (e.g. for language, or dismissed banners)? Show the idiom if so. The wizard needs to remember it was completed/skipped.
- **Bundling into the engine.**** Note (don't deeply read) scripts/bundle_ui.sh in the ENGINE repo C:\Users\avidu\Projects\bhi-p2b\scripts\bundle_ui.sh ? summarize how the built SPA gets vendored into the engine's backend/ui/ (so I know the post-build step), and whether it pins a khelsutra commit.

Return numbered findings 1-7 with file:line citations + the small snippets I asked for. Flag anything that makes the wizard tricky (e.g. no key-save endpoint exists in the engine, so the Gemini-key step must be guided-setup not auto-save; the 6-language parity requirement).

2. assistant (2026-06-22T03:18:37.157Z)

I'll map the khelsutra web SPA for your first-run wizard. Let me gather the key files systematically.

3. user (2026-06-22T03:18:37.640Z)

```
1      import { useEffect, useMemo, useState } from "react";
2      import { useTranslation, Trans } from "react-i18next";
3      import { useRallySelection } from "../hooks/useRallySelection";
4      import { RallyGrid } from "../components/RallyGrid";
5      import { QueryBar } from "../components/QueryBar";
6      import { TapFilters } from "../components/TapFilters";
7      import { SelectionFooter } from "../components/SelectionFooter";
8      import { LanguageSwitcher } from "../components/LanguageSwitcher";
9      import { CompileResult, type BuildState } from "../components/CompileResult";
10     import { IngestPanel } from "../components/IngestPanel";
11     import { runQuery, type ParsedQuery } from "../lib/query";
12     import { floorRisk } from "../lib/floor";
13     import { reelFilename } from "../lib/format";
14     import { classifyError } from "../lib/errors";
15     import { errorText } from "../lib/errorText";
16     import { log, newCorrelationId } from "../lib/log";
17     import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
from "../api/client";
18     import type { EngineConfig, RallySegment } from "../api/types";
19
20     /** One compact, structured description of how a query was understood (for the trace
log). */
21     function parseTrace(q: ParsedQuery) {
22         return {
23             raw: q.raw,
24             recognized: q.recognized,
25             filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
26             sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
27             limit: q.limit ?? null,
28             unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
29         };
30     }
31
32     // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
33     // contract; only honest fields are surfaced. A real session loads from POST /api/query.
34     const DEMO_RALLIES: RallySegment[] = [
35         { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
36         { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
37         { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
38         { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
39         { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
40         { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
41     ];
42
43     type LoadState =
44         | { status: "demo" }
45         | { status: "loading" }
```

```

46     | { status: "loaded"; count: number }
47     | { status: "error"; error: string };
48
49     export default function App() {
50         const { t, i18n } = useTranslation();
51         // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,
52         // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
53         // without a reload (it also rewrites ?video= so a refresh is stable).
54         const [videoId, setVideoId] = useState<string | null>()
55         () => new URLSearchParams(window.location.search).get("video"),
56         );
57
58         const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
59         const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
60         const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
61         const [reelName, setReelName] = useState("highlights");
62         const [build, setBuild] = useState<BuildState>({ status: "idle" });
63
64         // Display order is separate from selection: a query can re-sort the cards while the
shared
65         // selection set (sel) stays the single source of truth for what lands in the reel.
66         const sel = useRallySelection(rallies, "all");
67
68         // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
69         useEffect(() => {
70             if (!videoId) return;
71             const cid = newCorrelationId();
72             log("info", "rallies.load_start", { cid, video_id: videoId });
73             // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
74             // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
75             setLoadState({ status: "loading" });
76             let alive = true;
77             listRallies(videoId, cid)
78                 .then((rs) => {
79                     if (!alive) return;
80                     setRallies(rs);
81                     setOrder(rs);
82                     sel.apply(rs.map((r) => r.id));
83                     setLoadState({ status: "loaded", count: rs.length });
84                     log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
85                 })
86                 .catch((err) => {
87                     if (!alive) return;
88                     // Friendly, localized ? same shared mapping as build/ingest (A5): a load
failure on a slow or
89                     // down engine reads "is it running?", not a raw "ApiError: Network error
calling POST ?".
90                     const e = classifyError(err);
91                     setLoadState({ status: "error", error: errorText(t, e) });
92                     log("error", "rallies.load_failed", { cid, video_id: videoId, code: e.code,
detail: e.detail, error: String(err) });
93                 });
94             return () => {
95                 alive = false;
96             };
97             // videoId is stable for the page; sel.apply is a stable callback.
98             // eslint-disable-next-line react-hooks/exhaustive-deps
99         }, [videoId]);
100
101         // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is

```

```

102      // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
103      const [cfg, setCfg] = useState<EngineConfig | null>(null);
104      useEffect(() => {
105          let alive = true;
106          getConfig()
107              .then((c) => alive && setCfg(c))
108              .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
109          return () => {
110              alive = false;
111          };
112      }, []);
113      const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
114
115      // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
116      const ordinalOf = useMemo(() => {
117          const m = new Map<number, number>();
118          [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
119          return (id: number) => m.get(id) ?? 0;
120      }, [rallies]);
121
122      // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
123      const onBuild = async () => {
124          if (build.status === "building") return;
125          const cid = newCorrelationId();
126          const filename = reelFilename(reelName);
127          const ids = [...sel.selected];
128          if (!videoId) {
129              log("info", "build.no_video", { cid, selected: ids.length });
130              setBuild({
131                  status: "error",
132                  error: t("build.noVideo"),
133              });
134              return;
135          }
136          setBuild({ status: "building" });
137          log("info", "build.start", {
138              cid,
139              video_id: videoId,
140              filename,
141              selected: ids.length,
142              total_sec: Math.round(sel.totalSec),
143              below_floor: warning?.atRisk.length ?? 0,
144          });
145          try {
146              // Pass the active UI locale so the engine localizes the reel's branding watermark
(LW1).
147              const res = await compileReel(videoId, ids, filename, cid, i18n.resolvedLanguage
?? i18n.language);
148              const url = highlightUrl(videoId, filename);
149              setBuild({ status: "done", url, filename });
150              log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
151          } catch (err) {
152              // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads
"is the engine
153              // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500"
(A5).
154              const status = err instanceof ApiError ? err.status : undefined;
155              const e = classifyError(err);
156              setBuild({ status: "error", error: errorText(t, e) });
157              log("error", "build.failed", { cid, video_id: videoId, filename, status, code:
e.code, detail: e.detail, error: String(err) });

```

```

158     }
159     };
160
161     // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
162     // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
163     // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
164     const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
165         const cid = newCorrelationId();
166         log("info", "query.parse", { cid, source, ...parseTrace(q) });
167
168         // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
169         // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
170         if (q.unavailable.length > 0) {
171             log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
172         }
173
174         if (!q.recognized) {
175             log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
176             return; // empty / unparseable ? leave the user's curation untouched
177         }
178
179         try {
180             const { ordered, selectedIds } = runQuery(rallies, q);
181             setOrder(ordered);
182             sel.apply(selectedIds);
183             log("info", "query.apply", {
184                 cid,
185                 source,
186                 filters: q.filters.length,
187                 sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
188                 limit: q.limit ?? null,
189                 total: rallies.length,
190                 selected: selectedIds.length,
191                 selected_ids: selectedIds,
192                 dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
193             });
194             if (selectedIds.length === 0) {
195                 // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
196                 log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
197             }
198         } catch (err) {
199             // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
200             log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
201         }
202     };
203
204     // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
205     const onToggle = (id: number) => {
206         log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
207         sel.toggle(id);
208     };
209     const onBulk = (op: "select_all" | "clear" | "invert") => {
210         const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
211         log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:

```

```

resultCount });
212     if (op === "select_all") sel.selectAll();
213     else if (op === "clear") sel.clearAll();
214     else sel.invert();
215 };
216
217     return (
218         <main className="min-h-screen bg-paper text-ink pb-24">
219             <header className="bg-ink text-white">
220                 <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
221                     <span className="text-xl font-extrabold tracking-tight">
222                         Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
223                     </span>
224                     <LanguageSwitcher />
225                 </div>
226             </header>
227
228             <section className="mx-auto max-w-5xl px-6 py-10">
229                 <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
230                     {t("app.badge")}
231                 </span>
232                 <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
233                 <p className="mt-2 max-w-2xl text-ink/70">
234                     <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
235                 </p>
236
237                 {/* Front-half ingest (B-P2): when no video is loaded, point the tool at a
bucket URI / host
238                 path ? process ? load. Once a video_id exists, the existing back-half takes
over. */}
239                 {!videoId && (
240                     <IngestPanel
241                         onLoad={id => {
242                             setVideoId(id);
243                             window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
244                         }}
245                     />
246                 )}
247
248                 <ModeBanner load={loadState} videoId={videoId} />
249
250                 <div className="mt-6">
251                     <QueryBar onApply={onApplyQuery} />
252                     {/* A9: localized tap controls for users who can't type the English query
syntax. */}
253                     <TapFilters onApply={onApplyQuery} />
254                 </div>
255
256                 <div className="mt-6 flex flex-wrap gap-2 text-sm">
257                     <button type="button" onClick={() => onBulk("select_all")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.selectAll")}</button>
258                     <button type="button" onClick={() => onBulk("clear")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.clear")}</button>
259                     <button type="button" onClick={() => onBulk("invert")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.invert")}</button>
260                 </div>
261
262                 <div className="mt-5">

```

```

263         <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
264     </div>
265
266     <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
267 </section>
268
269     <SelectionFooter
270         count={sel.count}
271         totalSec={sel.totalSec}
272         reelName={reelName}
273         onReelNameChange={setReelName}
274         onBuild={onBuild}
275         building={build.status === "building"}
276         warning={warning}
277     />
278 </main>
279 );
280 }
281
282 function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
283     if (load.status === "demo") {
284         return (
285             <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
286                 <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
287             </p>
288         );
289     }
290     if (load.status === "loading") {
291         return (
292             <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
293                 <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code /> }} />
294             </p>
295         );
296     }
297     if (load.status === "error") {
298         return (
299             <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
text-red-800" data-testid="mode-banner">
300                 <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
components={{ code: <code /> }} />
301             </p>
302         );
303     }
304     return (
305         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
306             <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
<strong className="text-ink/70" /> }} />
307         </p>
308     );
309 }
310

```

4. user (2026-06-22T03:18:38.239Z)

```

1     // Typed client for the engine FastAPI. Default = SAME-ORIGIN relative `/api/*` (the
single-origin
2     // appliance reverse-proxies UI + engine behind one origin; in dev Vite proxies `/api` ?
:8000).
3     // Every call carries a correlation id (header + logs) so a journey is traceable end to
end.
4     import { log, newCorrelationId } from "../lib/log";
5     import type {
6         RallySegment,

```

```

7      QueryResponse,
8      CompileResponse,
9      EngineConfig,
10     ProcessResponse,
11     JobStatus,
12     UploadInitResponse,
13     UploadPartResponse,
14     UploadCompleteResponse,
15 } from "./types";
16
17 /** Resolve the engine API base. Default same-origin `"/api"`. For a SPLIT deploy ? SPA
on
18 * Cloudflare Pages + the engine on a separate `api.*` cloudflared tunnel
(HOSTING_PLAN.md $2) ?
19 * set `VITE_API_BASE` at build time to the engine's API base **including the `/api`
path**
20 * (e.g. `https://api.khelsutra.guru/api`), and lock the engine's
`RALLY_CORS_ALLOW_ORIGINS` to the
21 * Pages origin + put CF Access in front of both hostnames. A trailing slash is trimmed
so
22 * `` `${base}/process` `` never double-slashes. */
23 export function apiBase(): string {
24     return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
25 }
26
27 const BASE = apiBase();
28
29 export class ApiError extends Error {
30     readonly cid?: string;
31     readonly status?: number;
32     readonly body?: string;
33     constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
34         super(message);
35         this.name = "ApiError";
36         this.cid = opts.cid;
37         this.status = opts.status;
38         this.body = opts.body;
39         if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
40     }
41 }
42
43 interface RequestOpts {
44     method?: string;
45     body?: string;
46     headers?: Record<string, string>;
47     /** Correlation id to thread through this call; auto-generated if absent. */
48     cid?: string;
49 }
50
51 async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
52     const cid = opts.cid ?? newCorrelationId();
53     const url = `${BASE}${path}`;
54     const method = opts.method ?? "GET";
55     log("info", "api.request", { cid, method, url });
56
57     let res: Response;
58     try {
59         res = await fetch(url, {
60             method,
61             body: opts.body,
62             headers: { "content-type": "application/json", "x-correlation-id": cid,
...(opts.headers ?? {}) },
63         });
64     } catch (err) {

```



```

65         // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
66         log("error", "api.network_error", { cid, method, url, error: String(err) });
67         throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
68     }
69
70     if (!res.ok) {
71         const body = await res.text().catch(() => "");
72         log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
73         throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
74     }
75
76     const data = (await res.json()) as T;
77     log("info", "api.response", { cid, method, url, status: res.status });
78     return data;
79 }
80
81 /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
82 * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
fails 400
83 * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
84 export async function processVideo(videoPath: string, cid?: string, locale?: string):
Promise<ProcessResponse> {
85     // `locale` (the UI language) is recorded for PMF analytics (which markets use the
tool). Optional
86     // + omitted when absent (JSON.stringify drops undefined); the engine ignores it until
wired.
87     return request<ProcessResponse>("/process", {
88         method: "POST",
89         body: JSON.stringify({ video_path: videoPath, locale }),
90         cid,
91     });
92 }
93
94 /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
95 export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
96     return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
97 }
98
99 /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
100 export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
101     const data = await request<QueryResponse>("/query", {
102         method: "POST",
103         body: JSON.stringify({ video_id: videoId }),
104         cid,
105     });
106     return data.play_segments ?? [];
107 }
108
109 /** Stitch selected rallies into one reel. POST /api/compile {video_id, segment_ids,
output_filename} [main.py:342]. */
110 export async function compileReel(
111     videoId: string,
112     segmentIds: number[],
113     outputFilename: string,
114     cid?: string,
115     locale?: string,
116 ): Promise<CompileResponse> {
117     // `locale` (the UI language) localizes the reel's branding watermark engine-side.

```

```

Optional +
118     // omitted when absent; the engine falls back to the default watermark for an
unknown/absent locale.
119     return request<CompileResponse>("/compile", {
120         method: "POST",
121         body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename, locale }),
122         cid,
123     });
124 }
125
126 /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
127 export function highlightUrl(videoId: string, filename: string): string {
128     return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
129 }
130
131 /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
132 export async function getConfig(cid?: string): Promise<EngineConfig> {
133     return request<EngineConfig>("/config", { method: "GET", cid });
134 }
135
136 /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
137 export function minShotCount(cfg: EngineConfig): number | undefined {
138     return cfg.stitching?.min_shot_count;
139 }
140
141 // ?? chunked upload (backend/api/uploads.py) ?????????????????????????????????????????
142 // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
143 // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.
144
145 /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[uploads.py:64]. */
146 export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
147     return request<UploadInitResponse>("/upload/init", {
148         method: "POST",
149         body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
150         cid,
151     });
152 }
153
154 /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[uploads.py:89]. */
155 export async function uploadPart(
156     uploadId: string,
157     index: number,
158     chunk: Blob,
159     cid?: string,
160     signal?: AbortSignal,
161 ): Promise<UploadPartResponse> {
162     const c = cid ?? newCorrelationId();
163     const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
164     log("info", "api.request", { cid: c, method: "PUT", url });
165     let res: Response;
166     try {
167         res = await fetch(url, {
168             method: "PUT",
169             body: chunk,
170             headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
171             signal,

```

```

172     });
173     } catch (err) {
174         log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err)
});
175         throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
176     }
177     if (!res.ok) {
178         const body = await res.text().catch(() => "");
179         log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
body: body.slice(0, 500) });
180         throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
body });
181     }
182     log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
183     return (await res.json()) as UploadPartResponse;
184 }
185
186 /** Finalize: returns {path, video_id} ? feed `path` to processVideo() [uploads.py:123].
*/
187 export async function uploadComplete(uploadId: string, totalParts: number, cid?:
string): Promise<UploadCompleteResponse> {
188     return
request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
189         method: "POST",
190         body: JSON.stringify({ total_parts: totalParts }),
191         cid,
192     });
193 }
194
195 /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
[uploads.py:147]. */
196 export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
197     await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
cid });
198 }
199
200 export interface UploadProgress {
201     sentBytes: number;
202     totalBytes: number;
203 }
204
205 /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
complete.
206 * On ANY failure or abort it best-effort DELETES the server-side partial, then
rethrows so the
207 * caller can classify it (the engine's 400/413 detail survives). Returns {path,
video_id}.
208 * Note: a per-part 413 (over the per-part limit) aborts the upload server-side ?
recovery is a
209 * fresh init (re-pick the file), which the UI's retry does. */
210 export async function uploadFile(
211     file: File,
212     opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
= {},
213 ): Promise<UploadCompleteResponse> {
214     const cid = opts.cid ?? newCorrelationId();
215     const init = await uploadInit(file.name, file.size, cid);
216     // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math
to NaN (which
217     // would silently upload zero parts) ? fall back to the engine's 95 MB default.
218     const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ?
init.max_part_mb : 95;
219     const partBytes = partMb * 1024 * 1024;
220     const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
221     log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:

```

```

totalParts, part_mb: init.max_part_mb });
222     let sent = 0;
223     try {
224         for (let i = 0; i < totalParts; i++) {
225             if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
226             const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
227             await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
228             sent += chunk.size;
229             opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
230         }
231         const done = await uploadComplete(init.upload_id, totalParts, cid);
232         log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });
233         return done;
234     } catch (err) {
235         log("warn", "upload.aborting-partial", { cid, upload_id: init.upload_id, error:
String(err) });
236         await uploadAbort(init.upload_id, cid).catch(() => {});
237         throw err;
238     }
239 }
240

```

5. user (2026-06-22T03:18:38.886Z)

```

1     // Types mirror the engine's verified contract (badminton-highlight-indexer).
2     // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3     // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5     export interface RallyMetadata {
6         /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7         shot_count?: number;
8         /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9         shot_count_confidence?: string;
10        detector?: string;
11        [key: string]: unknown;
12    }
13
14    export interface RallySegment {
15        /** The integer the UI selects and passes to /api/compile as segment_ids. */
16        id: number;
17        start_time: number;
18        end_time: number;
19        duration: number;
20        /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21        server: string | null;
22        /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23        receiver: string | null;
24        metadata: RallyMetadata;
25        /** motion-only fabricated sub-events ? DO NOT surface. */
26        events?: unknown[];
27    }
28
29    export interface QueryResponse {
30        play_segments: RallySegment[];
31    }
32
33    export interface CompileResponse {
34        status: string;
35        /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
36        filepath: string;
37    }
38

```

```

39     export interface EngineConfig {
40         stitching?: {
41             /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
42             */
43             min_shot_count?: number;
44             [key: string]: unknown;
45         };
46         [key: string]: unknown;
47     }
48     /** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
49     /api/jobs/{job_id}. */
50     export interface ProcessResponse {
51         job_id: string;
52         /** Always "queued" at submit. */
53         status: string;
54         /** Relative poll path, e.g. "/api/jobs/<id>". */
55         poll: string;
56     }
57     /** Chunked upload (backend/api/uploads.py). The browser uploads a local file in
58     sequential parts
59     * ? max_part_mb, then `complete` returns a server-host `path` to feed straight into
60     POST /api/process. */
61     export interface UploadInitResponse {
62         upload_id: string;
63         /** Per-part ceiling in MB ? slice the file into parts this big. */
64         max_part_mb: number;
65         next_part: string;
66     }
67     export interface UploadPartResponse {
68         upload_id: string;
69         received_parts: number;
70         received_bytes: number;
71     }
72     export interface UploadCompleteResponse {
73         /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
74         path: string;
75         video_id: string;
76         size_bytes: number;
77     }
78     /** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
79     * `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
80     (success|failed).
81     * A `done` job whose `result.status === "failed"` (or with no video_id) is a success-
82     shaped
83     * FAILURE ? the UI must not treat it as a load. */
84     export interface JobStatus {
85         job_id: string;
86         status: "queued" | "running" | "done" | "failed";
87         /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
88         percentage. */
89         progress?: string | null;
90         /** The canonical video_id, surfaced top-level on success (also in result.video_id).
91         */
92         video_id?: string | null;
93         /** Execution error (worker crash / unexpected) when status === "failed". */
94         error?: string | null;
95         result?: { status?: string; video_id?: string; message?: string } | null;
96     }

```

No files found

7. assistant (2026-06-22T03:18:42.005Z)

Now let me search for the locale catalog files and test setup:

8. user (2026-06-22T03:18:43.556Z)

```
C:\Users\avidu\Projects\khelsutra\web\node_modules/i18next/.claude/settings.local.json
C:\Users\avidu\Projects\khelsutra\web\node_modules/i18next/dist/esm/package.json
C:\Users\avidu\Projects\khelsutra\web\node_modules/i18next/package.json
C:\Users\avidu\Projects\khelsutra\web\node_modules/i18next-browser-
  languagedetector/dist/esm/package.json
C:\Users\avidu\Projects\khelsutra\web\node_modules/i18next-browser-languagedetector/package.json
C:\Users\avidu\Projects\khelsutra\web\node_modules/i18next-browser-
  languagedetector/tsconfig.nonEsModuleInterop.json
C:\Users\avidu\Projects\khelsutra\web\node_modules/i18next-icu/dist/commonjs/package.json
C:\Users\avidu\Projects\khelsutra\web\node_modules/i18next-icu/package.json
C:\Users\avidu\Projects\khelsutra\web\node_modules/i18next-icu/tsconfig.json
C:\Users\avidu\Projects\khelsutra\web\node_modules/react-i18next/.eslintrc.json
C:\Users\avidu\Projects\khelsutra\web\node_modules/react-i18next/dist/es/package.json
C:\Users\avidu\Projects\khelsutra\web\node_modules/react-i18next/package.json
```

9. user (2026-06-22T03:18:43.920Z)

```
1      {
2        "name": "khelsutra-web",
3        "private": true,
4        "version": "0.0.0",
5        "type": "module",
6        "description": "KhelSutra Studio ? operator console SPA (drives the rally indexer
engine).",
7        "scripts": {
8          "dev": "vite",
9          "build": "vite build",
10         "preview": "vite preview",
11         "typecheck": "tsc --noEmit",
12         "test": "vitest run",
13         "test:coverage": "vitest run --coverage"
14       },
15       "dependencies": {
16         "@fontsource-variable/noto-sans-devanagari": "^5.2.8",
17         "@fontsource-variable/noto-sans-kannada": "^5.2.9",
18         "i18next": "^26.3.1",
19         "i18next-browser-languagedetector": "^8.2.1",
20         "i18next-icu": "^2.4.3",
21         "react": "^19.0.0",
22         "react-dom": "^19.0.0",
23         "react-i18next": "^17.0.8"
24       },
25       "devDependencies": {
26         "@tailwindcss/vite": "^4.0.0",
27         "@testing-library/dom": "^10.4.1",
28         "@testing-library/react": "^16.3.2",
29         "@types/node": "^26.0.0",
30         "@types/react": "^19.0.0",
31         "@types/react-dom": "^19.0.0",
32         "@vitejs/plugin-react": "^4.3.4",
33         "@vitest/coverage-v8": "^3.2.6",
34         "jsdom": "^29.1.1",
35         "tailwindcss": "^4.0.0",
36         "typescript": "^5.7.0",
37         "vite": "^6.0.0",
38         "vitest": "^3.0.0"
```

```
39     }
40   }
41
```

10. user (2026-06-22T03:18:44.864Z)

```
C:\Users\avidu\Projects\kheIsutra\web\src\components\IngestPanel.test.tsx
C:\Users\avidu\Projects\kheIsutra\web\src\components\IngestPanel.tsx
```

11. assistant (2026-06-22T03:18:46.788Z)

Now let me check the locale structure and IngestPanel:

12. user (2026-06-22T03:18:47.905Z)

(Bash completed with no output)

13. user (2026-06-22T03:18:48.473Z)

```
Exit code 2
/usr/bin/bash: eval: line 1: unexpected EOF while looking for matching `''
```

14. assistant (2026-06-22T03:18:50.205Z)

Let me use a different approach:

15. user (2026-06-22T03:18:51.208Z)

No files found

16. user (2026-06-22T03:18:51.482Z)

```
web\src\components\IngestPanel.test.tsx
web\src\components\A4.touch.test.tsx
web\src\components\upload.test.tsx
web\src\i18n\i18n.test.tsx
web\src\components\TapFilters.test.tsx
web\src\components\build-error.test.tsx
web\src\components\QueryBar.test.tsx
web\src\components\SelectionFooter.test.tsx
web\src\components\A8.status.test.tsx
```

17. user (2026-06-22T03:18:54.094Z)

```
1    // The front-half ingest screen (appliance S1/S2; tracker B-P2 + T4). Two ways to point
the tool at a
2    // video: UPLOAD a local file (primary ? alpha users have a file, not a URI; chunked to
the engine's
3    // /api/upload/* endpoints) or paste a `gs://` bucket URI / host path (secondary). Both
converge on
4    // the SAME back-half: POST /api/process ? poll GET /api/jobs/{id} ? hand the video_id
to App. Honest
5    // by construction: the engine reports coarse phase WORDS (not a %), and a success-
shaped failure
6    // (done but no rallies) is shown, never silently treated as a load.
7    import { useState } from "react";
8    import type { ChangeEvent, FormEvent } from "react";
9    import { useTranslation } from "react-i18next";
10   import { useIngestJob } from "../hooks/useIngestJob";
11   import { useUpload } from "../hooks/useUpload";
12   import { errorText } from "../lib/errorText";
13   import { StatusChip, Spinner } from "../StatusChip";
14
```

```

15     interface Props {
16         /** Called with the engine's video_id once a submitted job finishes successfully. */
17         onLoaded: (videoId: string) => void;
18     }
19
20     export function IngestPanel({ onLoaded }: Props) {
21         const { t, i18n } = useTranslation();
22         const [uri, setUri] = useState("");
23         const { phase, submit, cancel: cancelIngest, reset: resetIngest, busy: ingestBusy } =
useIngestJob(onLoaded);
24         // The UI locale recorded with the submission (PMF analytics + the localized reel
watermark).
25         const locale = i18n.resolvedLanguage ?? i18n.language;
26         // On upload completion, feed the returned server PATH straight into the engine
processing back-half.
27         const upload = useUpload((path) => submit(path, locale));
28         const busy = ingestBusy || upload.busy;
29
30         const onSubmitUri = (e: FormEvent) => {
31             e.preventDefault();
32             submit(uri, locale);
33         };
34         const onFile = (e: ChangeEvent<HTMLInputElement>) => {
35             const file = e.target.files?.[0];
36             if (file) upload.start(file);
37             e.target.value = ""; // let re-picking the same file fire change again
38         };
39         const cancelAll = () => {
40             upload.cancel();
41             cancelIngest();
42         };
43         const resetAll = () => {
44             upload.reset();
45             resetIngest();
46         };
47
48         // Shared mapping with the build flow (lib/errorText): an `engine` failure carries the
engine's own
49         // message (a 400 scheme/path/extension detail, a worker error); every other code maps
to a friendly
50         // localized line. Surface whichever half failed (upload or engine) ? they're mutually
exclusive.
51         const errorState = upload.phase.status === "error" ? upload.phase.error : phase.status
=== "error" ? phase.error : null;
52         const errorMsg = errorState ? errorText(t, errorState) : null;
53
54         return (
55             <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
56                 <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
57                 <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
58
59                 {/* PRIMARY: upload a local file (T4). A native file input, visually a button
(?44px, A4). */}
60                 <div className="mt-4">
61                     <label
62                         className={
63                             // focus-within makes the visible button show a focus ring when the (sr-
only) input is
64                             // focused ? a keyboard user needs a visible focus indicator (WCAG 2.4.7).
65                             "inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2 text-sm font-
semibold text-white hover:bg-ink2 focus-within:outline-none focus-within:ring-2 focus-
within:ring-green focus-within:ring-offset-2 " +
66                             (busy ? "pointer-events-none opacity-40" : "cursor-pointer")
67                         }

```



```

68         >
69         {t("ingest.chooseFile")}
70         <input
71             type="file"
72             accept=".mp4,.mov,video/mp4,video/quicktime"
73             aria-label={t("ingest.uploadAria")}
74             disabled={busy}
75             onChange={onFile}
76             className="sr-only"
77         />
78     </label>
79     <p className="mt-2 text-xs text-ink/40">{t("ingest.fileHint")}</p>
80 </div>
81
82     {/* SECONDARY: or point at a cloud bucket / host path. */}
83     <div className="mt-4 border-t border-black/5 pt-4">
84         <p className="text-xs font-semibold uppercase tracking-wide text-ink/40">{t("ingest.orLink")}</p>
85         <form onSubmit={onSubmitUri} className="mt-2 flex flex-wrap items-center gap-2">
86             <input
87                 type="text"
88                 value={uri}
89                 onChange={(e) => setUri(e.target.value)}
90                 placeholder={t("ingest.uriPlaceholder")}
91                 aria-label={t("ingest.uriAriaLabel")}
92                 disabled={busy}
93                 className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
94             />
95             <button
96                 type="submit"
97                 disabled={busy}
98                 className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-white disabled:opacity-40"
99             >
100                 {phase.status === "submitting" ? t("ingest.submitting") :
t("ingest.submit")}
101             </button>
102         </form>
103         <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
104     </div>
105
106     {/* Live status (aria-live so screen readers hear progress) as icon+text chips ?
never icon-only
107     (A8). Upload shows a determinate % bar; the engine phase is indeterminate
(phase words). */}
108     <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
109         {upload.phase.status === "uploading" && (
110             <div data-testid="upload-progress">
111                 {/* Static chip label (not the per-tick %) so the aria-live region announces
once; the live
112                 % rides the progressbar's aria-valuenow + a sighted-only label (A8/T4).
*/}
113                 <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.uploadingStatus")} />
114                 <div className="mt-2 flex max-w-sm items-center gap-2">
115                     <div
116                         className="h-1.5 flex-1 overflow-hidden rounded-full bg-black/10"
117                         role="progressbar"
118                         aria-label={t("ingest.uploadProgress")}
119                         aria-valuenow={upload.phase.pct}
120                         aria-valuemin={0}
121                         aria-valuemax={100}
122                     >
123                     <div className="h-full bg-green transition-all" style={{ width:

```

```

`${upload.phase.pct}%` }} />
124         </div>
125         <span aria-hidden="true" className="text-xs tabular-nums text-ink/50">
126             {upload.phase.pct}%
127         </span>
128     </div>
129     <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
130     <button
131         type="button"
132         onClick={cancelAll}
133         className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
134     >
135         {t("ingest.cancel")}
136     </button>
137 </div>
138 }}
139 {ingestBusy && (
140     <div>
141         <StatusChip
142             tone="info"
143             icon={<Spinner />}
144             label={
145                 t("ingest.working") +
146                 (phase.status === "polling" && phase.progress
147                     ? ` · ${t("ingest.progress", { progress: phase.progress })}`
148                     : "")
149             }
150         />
151         /* Reassurance for slow uplinks: the job runs on the server, so it's safe
to wait. */}
152         <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
153         <button
154             type="button"
155             onClick={cancelAll}
156             className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
157         >
158             {t("ingest.cancel")}
159         </button>
160     </div>
161 }}
162 {errorMsg && (
163     <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
164         <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />
165         <p className="mt-2">{errorMsg}</p>
166         <button
167             type="button"
168             onClick={resetAll}
169             className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
170         >
171             {t("ingest.retry")}
172         </button>
173     </div>
174 }}
175 </div>
176 </section>
177 );
178 }
179

```

```

1      // @vitest-environment jsdom
2      //
3      // The ingest screen end to end: type a URI ? submit ? (mocked engine) ? onLoaded gets
the video_id;
4      // and the two failure surfaces a non-technical user actually hits ? the engine's own
400 detail,
5      // and a friendly "is the engine running?" when it's unreachable. Mounts the real i18n
catalog.
6      import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
7      import { render, screen, fireEvent, cleanup, act } from "@testing-library/react";
8      import i18n from "../i18n/index";
9      import { IngestPanel } from "../IngestPanel";
10     import type { JobStatus } from "../api/types";
11
12     type FetchFn = typeof fetch;
13
14     function httpOk(status: number, json: unknown) {
15         return { ok: status >= 200 && status < 300, status, json: async () => json, text:
async () => JSON.stringify(json) };
16     }
17
18     function fetchSeq(process: unknown, jobs: JobStatus[]): FetchFn {
19         let i = 0;
20         return vi.fn(async (url: string | URL, init?: RequestInit) => {
21             const u = String(url);
22             if (u === "/api/process" && (init?.method ?? "GET") === "POST") return process;
23             if (u.startsWith("/api/jobs/")) {
24                 const body = jobs[Math.min(i, jobs.length - 1)];
25                 i += 1;
26                 return httpOk(200, body);
27             }
28             throw new Error(`unexpected fetch ${u}`);
29         }) as unknown as FetchFn;
30     }
31
32     beforeEach(async () => {
33         window.localStorage.clear();
34         await i18n.changeLanguage("en"); // guarantees init completed + a known locale for the
assertions
35     });
36
37     afterEach(() => {
38         cleanup();
39         vi.restoreAllMocks();
40         vi.unstubAllGlobals();
41         vi.useRealTimers();
42     });
43
44     const typeUri = (value: string) =>
45         fireEvent.change(screen.getByLabelText(/video location/i), { target: { value } });
46     const clickProcess = () => fireEvent.click(screen.getByRole("button", { name: /process
video/i }));
47
48     describe("IngestPanel", () => {
49         it("submits a URI, polls, and hands the video_id to onLoaded", async () => {
50             vi.useFakeTimers();
51             vi.stubGlobal(
52                 "fetch",
53                 fetchSeq(httpOk(202, { job_id: "j", status: "queued", poll: "/api/jobs/j" }), [
54                     { job_id: "j", status: "running", progress: "segmenting" },
55                     { job_id: "j", status: "done", video_id: "vidZ", result: { status: "success",
video_id: "vidZ" } } ],
56                 ),
57             );
58             const onLoaded = vi.fn();

```

```

59     render(<IngestPanel onLoaded={onLoaded} />);
60     typeUri("gs://bucket/match.mp4");
61     clickProcess();
62
63     await act(async () => {
64       await vi.advanceTimersByTimeAsync(0); // flush submit ? start polling
65     });
66     await act(async () => {
67       await vi.advanceTimersByTimeAsync(1000); // running tick
68     });
69     await act(async () => {
70       await vi.advanceTimersByTimeAsync(1500); // done tick
71     });
72     expect(onLoaded).toHaveBeenCalled("vidZ");
73   });
74
75   it("shows the engine's own 400 detail (e.g. an unsupported scheme)", async () => {
76     vi.stubGlobal("fetch", fetchSeq(httpOk(400, { detail: "unsupported storage URI
scheme: ftp://" }), []));
77     render(<IngestPanel onLoaded={vi.fn()} />);
78     typeUri("ftp://nope");
79     clickProcess();
80     expect(await screen.findByText(/unsupported storage URI scheme/i)).toBeTruthy();
81   });
82
83   it("shows a friendly, localized message when the engine is unreachable", async () => {
84     vi.stubGlobal("fetch", vi.fn(async () => {
85       throw new Error("offline");
86     }) as unknown as FetchFn);
87     render(<IngestPanel onLoaded={vi.fn()} />);
88     typeUri("gs://bucket/match.mp4");
89     clickProcess();
90     expect(await screen.findByText(/can't reach khelsutra/i)).toBeTruthy();
91   });
92
93   it("rejects an empty URI with a hint and never calls the engine", () => {
94     const f = vi.fn();
95     vi.stubGlobal("fetch", f);
96     render(<IngestPanel onLoaded={vi.fn()} />);
97     clickProcess();
98     expect(screen.getByTestId("ingest-error")).toBeTruthy();
99     expect(f).not.toHaveBeenCalled();
100   });
101 });
102

```

19. user (2026-06-22T03:18:56.066Z)

C:\Users\avidu\Projects\khelsutra\web\package-lock.json
C:\Users\avidu\Projects\khelsutra\web\package.json
C:\Users\avidu\Projects\khelsutra\web\tsconfig.json

20. user (2026-06-22T03:18:59.452Z)

C:\Users\avidu\Projects\khelsutra\web\src\i18n\i18n.test.tsx

21. user (2026-06-22T03:18:59.739Z)

```

1    // @vitest-environment jsdom
2    //
3    // The SPA had NO render tests before this (verified) ? this file adds the jsdom safety
net for the
4    // i18n scaffold (tracker B1): real copy renders from the catalog, the switcher persists
the choice,

```

```

5      // missing-locale falls back to English, and ICU plurals format. It mounts the real <App
/>.
6      import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";
7      import { render, screen, cleanup, fireEvent } from "@testing-library/react";
8      import i18n, { LANG_STORAGE_KEY } from "../index";
9      import App from "../App";
10
11      beforeEach(async () => {
12          // No engine in the test env: stub fetch so App's getConfig()/listRallies effects
reject quietly
13          // (App handles this gracefully ? cfg stays null) instead of hitting the network.
14          vi.stubGlobal("fetch", vi.fn(() => Promise.reject(new Error("no engine in test"))));
15          window.localStorage.clear();
16          await i18n.changeLanguage("en"); // also guarantees init completed before the first
render
17      });
18
19      afterEach(() => {
20          cleanup();
21          vi.unstubAllGlobals();
22      });
23
24      describe("i18n scaffold (B1)", () => {
25          it("renders user-facing copy from the en catalog (not raw keys)", () => {
26              render(<App />);
27              expect(screen.getByRole("heading", { level: 1 })).textContent.toBe("Pick the rallies
for your reel");
28              expect(screen.getByRole("button", { name: "Select all" })).toBeTruthy();
29          });
30
31          it("B2: the extracted component strings render (no leftover raw keys across the
page)", () => {
32              const { container } = render(<App />);
33              expect(screen.getByRole("button", { name: "Apply" })).toBeTruthy(); // QueryBar
34              expect(screen.getByRole("button", { name: /Build reel/ })).toBeTruthy(); //
SelectionFooter
35              expect(container.textContent).toContain("rally length"); // <Trans> hero intro +
query support
36              expect(container.textContent).toMatch(/\d+ rallies selected · \d+:\d+ total/); //
ICU footer summary
37              expect(container.textContent).toMatch(/Rally \d+/); // RallyGrid card
38              // No untranslated catalog keys leaked into the DOM.
39              expect(container.textContent).not.toMatch(/\b(app|grid|footer|query|mode|result|buil
d)\.[a-zA-Z]+/);
40          });
41
42          it("offers a language switcher and persists the choice (persisted pref wins on next
visit)", () => {
43              render(<App />);
44              const knBtn = screen.getByRole("button", { name: "?????" });
45              fireEvent.click(knBtn);
46              expect(i18n.language).toBe("kn");
47              expect(window.localStorage.getItem(LANG_STORAGE_KEY)).toBe("kn");
48              expect(knBtn.getAttribute("aria-pressed")).toBe("true"); // highlights the chosen
locale
49              // Switching actually re-renders the page into Kannada (the kn catalog, B5).
50              expect(screen.getByRole("heading", { level: 1 })).textContent.toBe("????? ?????????
????????????? ?????");
51          });
52
53          it("B5: switching to Kannada renders the kn catalog (Kannada copy, not English)",
async () => {
54              await i18n.changeLanguage("kn");
55              render(<App />);
56              expect(screen.getByRole("heading", { level: 1 })).textContent.toBe("????? ?????????

```

```

???????????? ????");
57     expect(screen.getByRole("button", { name: "???????" })).toBeTruthy(); // Apply ?
Kannada
58     expect(screen.getByRole("button", { name: "????????? ????????" })).toBeTruthy(); //
Select all ? Kannada
59     expect(document.documentElement.lang).toBe("kn"); // <html lang> tracks the active
locale (a11y)
60     });
61
62     it("B6: switching to Hindi renders the hi catalog (extensibility proof ? just a
catalog)", async () => {
63         await i18n.changeLanguage("hi");
64         render(<App />);
65         expect(screen.getByRole("heading", { level: 1 }).textContent).toBe("???? ?? ? ? ? ?
?????? ????");
66         expect(screen.getByRole("button", { name: "???? ???? " })).toBeTruthy(); // Apply ?
Hindi
67         expect(document.documentElement.lang).toBe("hi");
68     });
69
70     it("a locale with no catalog falls back to English (never a raw key)", async () => {
71         await i18n.changeLanguage("ta"); // ta has no catalog
72         render(<App />);
73         expect(screen.getByRole("heading", { level: 1 }).textContent).toBe("Pick the rallies
for your reel");
74     });
75
76     it("formats ICU plurals from the catalog", () => {
77         expect(i18n.t("selection.count", { count: 1 })).toBe("1 rally selected");
78         expect(i18n.t("selection.count", { count: 3 })).toBe("3 rallies selected");
79     });
80 });
81
82 // The owner's bar (#41): a user whose BROWSER/OS language is Kannada/Hindi must auto-
get the
83 // respective version. These prove the negotiation NOW at the detection level (B1, en-
only); the
84 // kn/hi CONTENT assertions ride the catalogs in B5/B6. See FRIENDLY_AND_MULTILINGUAL.md
$6.5 (N1?N9).
85 describe("browser-language negotiation (B1: N1/N3?N7)", () => {
86     const detectBases = () => {
87         const d = i18n.services.languageDetector.detect();
88         return (Array.isArray(d) ? d : [d]).filter(Boolean).map((l) =>
String(l).split("-")[0]);
89     };
90
91     it("N1/N5: a Kannada browser (kn-IN) is auto-detected as kn for a first-time visitor",
() => {
92         window.localStorage.clear(); // no persisted preference yet
93         vi.stubGlobal("navigator", { language: "kn-IN", languages: ["kn-IN", "en-IN"] });
94         expect(detectBases()).toContain("kn"); // the kn-IN region subtag normalizes to base
kn
95     });
96
97     it("N2: a Hindi browser (hi-IN) is auto-detected as hi (now a supported locale)", ()
=> {
98         window.localStorage.clear();
99         vi.stubGlobal("navigator", { language: "hi-IN", languages: ["hi-IN", "en-IN"] });
100         expect(detectBases()).toContain("hi");
101     });
102
103     it("N4: a persisted preference beats the browser language (returning user keeps their
choice)", () => {
104         window.localStorage.setItem(LANG_STORAGE_KEY, "kn");
105         vi.stubGlobal("navigator", { language: "en-US", languages: ["en-US"] });

```

```

106     expect(detectBases()[0]).toBe("kn");
107 });
108
109 it("N3: persisted pref and ?lang are negotiated BEFORE the browser language", () => {
110     const order = i18n.options.detection?.order ?? [];
111     expect(order.indexOf("localStorage")).toBeGreaterThanOrEqual(0);
112     expect(order.indexOf("localStorage")).toBeLessThan(order.indexOf("navigator"));
113     expect(order.indexOf("querystring")).toBeLessThan(order.indexOf("navigator"));
114 });
115
116 it("N6: an unsupported browser language (Tamil) serves English content ? never a
locale we can't render", async () => {
117     window.localStorage.clear();
118     await i18n.changeLanguage("ta"); // ta is not in SUPPORTED_LOCALES
119     render(<App />);
120     expect(screen.getByRole("heading", { level: 1 }).textContent).toBe("Pick the rallies
for your reel");
121 });
122
123 it("N7: a key with no catalog entry falls back to English (no raw key)", async () => {
124     await i18n.changeLanguage("ta"); // ta has no catalog ? keys resolve via
fallbackLng=en
125     expect(i18n.t("app.title")).toBe("Pick the rallies for your reel");
126     expect(i18n.t("bulk.selectAll")).toBe("Select all");
127 });
128 });
129
130 // MULTILINGUAL_EXPANSION Phase A ? es/da/id render + auto-negotiate, same rigor as
en/kn/hi.
131 // es is the only authoring wrinkle (CLDR 3-form {one, many, other}); da is 2-form; id
is single-form.
132 describe("MULTILINGUAL Phase A (es/da/id)", () => {
133     it("renders each new locale's catalog (real translations, not English / not raw
keys)", async () => {
134         const titles: Record<string, string> = {
135             es: "Elige las jugadas para tu reel",
136             da: "Vælg duellerne til din reel",
137             id: "Pilih reli untuk reel kamu",
138         };
139         for (const [loc, title] of Object.entries(titles)) {
140             await i18n.changeLanguage(loc);
141             cleanup();
142             const { container } = render(<App />);
143             expect(screen.getByRole("heading", { level: 1 }).textContent, loc).toBe(title);
144             expect(container.textContent,
loc).not.toMatch(/\b(app|grid|footer|query|ingest)\.[a-zA-Z]+/); // no raw keys
145         }
146         await i18n.changeLanguage("en");
147     });
148
149     it("es uses the 3-form plural (singular vs plural word forms)", async () => {
150         await i18n.changeLanguage("es");
151         expect(i18n.t("selection.count", { count: 1 })).toBe("1 jugada seleccionada"); //
one
152         expect(i18n.t("selection.count", { count: 2 })).toBe("2 jugadas seleccionadas"); //
other
153         await i18n.changeLanguage("en");
154     });
155
156     it("id is single-form (only an 'other' arm) and still interpolates the count", async
() => {
157         await i18n.changeLanguage("id");
158         expect(i18n.t("selection.count", { count: 1 })).toBe("1 reli dipilih");
159         expect(i18n.t("selection.count", { count: 5 })).toBe("5 reli dipilih");
160         await i18n.changeLanguage("en");

```

```

161     });
162
163     it("auto-negotiates es/da/id from the browser language (region subtags normalize to
base)", () => {
164         for (const [lng, base] of [["es-ES", "es"], ["da-DK", "da"], ["id-ID", "id"]] as
const) {
165             window.localStorage.clear();
166             vi.stubGlobal("navigator", { language: lng, languages: [lng] });
167             const d = i18n.services.languageDetector.detect();
168             const bases = (Array.isArray(d) ? d : [d]).filter(Boolean).map((l) =>
String(l).split("-")[0]);
169             expect(bases, lng).toContain(base);
170         }
171     });
172 });
173

```

22. assistant (2026-06-22T03:19:01.809Z)

Now let me find the catalog files and the parity test:

23. user (2026-06-22T03:19:03.267Z)

(Bash completed with no output)

24. user (2026-06-22T03:19:04.078Z)

(Bash completed with no output)

25. user (2026-06-22T03:19:06.156Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\kheIsutra.
Did you mean index.ts?

26. user (2026-06-22T03:19:09.220Z)

Exit code 2

/usr/bin/bash: eval: line 1: unexpected EOF while looking for matching `''

27. user (2026-06-22T03:19:11.415Z)

```

web\src\i18n\i18n.test.tsx
web\src\i18n\hygiene.test.ts
web\src\i18n\index.ts
web\src\i18n\catalog-invariants.test.ts

```

28. user (2026-06-22T03:19:13.796Z)

```

1      // i18n scaffold (tracker B1) ? the shared localization foundation for KheIsutra Studio.
2      //
3      // Per docs/FRIENDLY_AND_MULTILINGUAL.md + the engine's I18N_PLAN.md:
4      //   - keys, never prose (t("app.title"), enforced later by a no-bare-strings lint, B2);
5      //   - ICU MessageFormat for interpolation/plurals (Kannada/Hindi plural rules live in
the catalog,
6      //     not the code ? replaces the SPA's hand-rolled `count > 1 ? "s" : ""`);
7      //   - one negotiation order shared with the marketing site (B3);
8      //   - always-fallback to `en` so a missing translation degrades to English, never a raw
key.
9      import i18n from "i18next";
10     import { initReactI18next } from "react-i18next";
11     import LanguageDetector from "i18next-browser-languagedetector";
12     import ICU from "i18next-icu";
13     // Catalogs are HOISTED to repo-root /locales (D-CATALOG-HOME) ? the single source of
truth shared

```



```

14      // by this SPA and the marketing site's t() shim (tracker B3). Vite resolves the
workspace root via
15      // the repo `.git` marker, so importing the parent-dir /locales works in dev, build, and
viteest.
16      import enCommon from "../../locales/en/common.json";
17      import knCommon from "../../locales/kn/common.json";
18      import hiCommon from "../../locales/hi/common.json";
19      import esCommon from "../../locales/es/common.json";
20      import daCommon from "../../locales/da/common.json";
21      import idCommon from "../../locales/id/common.json";
22
23      // Supported locales: en/kn/hi foundation (FRIENDLY D6) + MULTILINGUAL_EXPANSION Phase A
(es/da/id,
24      // Latin, catalog-only). Anything else falls back to `en`.
25      export const SUPPORTED_LOCALES = ["en", "kn", "hi", "es", "da", "id"] as const;
26      export type Locale = (typeof SUPPORTED_LOCALES)[number];
27      export const DEFAULT_LOCALE: Locale = "en";
28
29      // ONE persisted-preference key, shared with the marketing site's t() shim (D9 / tracker
B3), so a
30      // visitor's language choice carries across both surfaces.
31      export const LANG_STORAGE_KEY = "khelsutra.lang";
32
33      // Catalogs live in repo-root /locales (D-CATALOG-HOME), consumed by both surfaces. `kn`
+ `hi`
34      // shipped (B5/B6, AI-drafted ? native review pending). A missing key in any locale
still falls back
35      // to `en` ? the by-design invariant.
36      const resources = {
37          en: { common: enCommon },
38          kn: { common: knCommon },
39          hi: { common: hiCommon },
40          es: { common: esCommon },
41          da: { common: daCommon },
42          id: { common: idCommon },
43      } as const;
44
45      void i18n
46          .use(ICU)
47          .use(LanguageDetector)
48          .use(initReactI18next)
49          .init({
50              resources,
51              fallbackLng: DEFAULT_LOCALE,
52              supportedLngs: [...SUPPORTED_LOCALES],
53              nonExplicitSupportedLngs: true, // en-IN -> en
54              defaultNS: "common",
55              ns: ["common"],
56              interpolation: { escapeValue: false }, // React already escapes
57              returnNull: false,
58              // Negotiation order (I18N_PLAN §2.3): persisted pref -> ?lang= -> browser -> en.
59              detection: {
60                  order: ["localStorage", "querystring", "navigator"],
61                  lookupLocalStorage: LANG_STORAGE_KEY,
62                  lookupQuerystring: "lang",
63                  caches: ["localStorage"],
64              },
65              react: { useSuspense: false },
66          });
67
68      // Keep <html lang> in sync with the active locale (screen readers + SEO use it).
Reflects the
69      // resolved content language, so an `en` fallback for an absent locale still reads as
`en`.
70      function syncDocumentLang(lng: string) {

```

```

71     if (typeof document !== "undefined") {
72         document.documentElement.lang = (i18n.resolvedLanguage || lng ||
DEFAULT_LOCALE).split("-")[0];
73     }
74 }
75 i18n.on("languageChanged", syncDocumentLang);
76 syncDocumentLang(i18n.language ?? DEFAULT_LOCALE);
77
78 export default i18n;
79

```

29. user (2026-06-22T03:19:14.620Z)

```

1     // Content regression guards for the localization catalogs ? the rules the adversarial
translation
2     // audit (#77) established, pinned so the specific bugs can never silently return. This
complements
3     // hygiene.test.ts (which proves key/placeholder PARITY) by asserting catalog CONTENT.
Each block maps
4     // to a concrete #77 bug class; the mutation that reintroduces the bug fails here. Pure
data assertions
5     // over repo-root /locales (D-CATALOG-HOME) ? no render context needed.
6     import { describe, it, expect } from "vitest";
7     import { readFileSync, readdirSync } from "node:fs";
8     import { join } from "node:path";
9
10    const LOCALES_DIR = join(import.meta.dirname, "..", "..", "..", "locales");
11
12    type Flat = Record<string, string>;
13    function flatEntries(obj: Record<string, unknown>, prefix = ""): Flat {
14        const out: Flat = {};
15        for (const [k, v] of Object.entries(obj)) {
16            const key = prefix ? `${prefix}.${k}` : k;
17            if (v && typeof v === "object" && !Array.isArray(v)) Object.assign(out,
flatEntries(v as Record<string, unknown>, key));
18            else out[key] = String(v);
19        }
20        return out;
21    }
22    const load = (loc: string): Flat =>
23        flatEntries(JSON.parse(readFileSync(join(LOCALES_DIR, loc, "common.json"), "utf8")) as
Record<string, unknown>);
24
25    const ALL_LOCALES = readdirSync(LOCALES_DIR, { withFileTypes: true })
26        .filter((d) => d.isDirectory())
27        .map((d) => d.name);
28    const en = load("en");
29
30    // (1) Internal-token / debug-tag leaks ? EVERY locale. The engine's internal
`min_shot_count` config
31    // name leaked into footer.floorSomeRest (#77, kn even wrapped it in <code>). No
developer/debug tag
32    // (kbd/samp/tt/var) or that token may reach any user-facing string. <code> is likewise
banned ?
33    // EXCEPT on the marketing site's developer "own your model" page (site.ownModel.*),
which intentionally
34    // renders inline <code> (the rally_annotator.lua filename, the ending_reason enum,
<video>.rallies.csv)
35    // through the site shim's data-i18n-html form. That namespace is the sole, deliberate
exception; the
36    // SPA console keys (where the #77 leak happened) still fail on any <code>.
37    describe("catalog content ? no internal token / debug-tag leaks (every locale)", () => {
38        const TOKEN = /min_shot_count/;
39        const DEBUG_TAG = /<\/?(?:kbd|samp|tt|var)\b/i;
40        const CODE_TAG = /<\/?code\b/i;

```

```

41     const codeAllowed = (key: string) => key.startsWith("site.ownModel.");
42     for (const loc of ALL_LOCALES) {
43         it(`${loc}: no 'min_shot_count' token or stray <code>/<kbd>/<samp>/<tt>/<var> tag`,
44         () => {
45             const offenders = Object.entries(load(loc))
46                 .filter(([k, v]) => TOKEN.test(v) || DEBUG_TAG.test(v) || (CODE_TAG.test(v) &&
47                 !codeAllowed(k)))
48                 .map(([k]) => k);
49             expect(offenders).toEqual([]);
50         });
51     }
52     // (2) Brand ? never transliterated, and present (Latin) on EVERY key en uses it on. The
53     // positive rule
54     // is the real guard: self-maintaining (auto-covers future brand keys, incl. errors.* /
55     // ingest.hint that
56     // the old 3-key denylist missed) and it catches ANY transliteration spelling at once ?
57     // a transliterated
58     // value simply won't contain Latin "KheḷSutra". The Indic-spelling denylist is cheap
59     // defence-in-depth.
60     describe("catalog content ? the brand stays Latin 'KheḷSutra' everywhere en uses it", () => {
61         const brandKeys = Object.keys(en).filter((k) => en[k].includes("KheḷSutra"));
62         it("en actually carries the brand on several keys (so the guard is meaningful)", () => {
63             expect(brandKeys.length).toBeGreaterThanOrEqual(6);
64         });
65         for (const loc of ["kn", "hi"] as const) {
66             it(`${loc}: Latin "KheḷSutra" is present on every key whose en value uses it`, () => {
67                 const e = load(loc);
68                 const missing = brandKeys.filter((k) => !(e[k] ?? "").includes("KheḷSutra"));
69                 expect(missing).toEqual([]);
70             });
71         }
72         const TRANSLIT: Record<string, string> = { kn: "?????????", hi: "?????????" };
73         for (const loc of ["kn", "hi"] as const) {
74             it(`${loc}: the known Indic transliteration of the brand never appears`, () => {
75                 const offenders = Object.entries(load(loc))
76                     .filter(([k, v]) => v.includes(TRANSLIT[loc]))
77                     .map(([k]) => k);
78                 expect(offenders).toEqual([]);
79             });
80         }
81     });
82     // (3) Engine/server/bucket/backend jargon only where en uses it (kn/hi). Self-
83     // maintaining: a localized
84     // jargon term ? the Latin word OR an Indic transliteration ? is allowed ONLY on a key
85     // whose en value
86     // also uses the English word. en avoids these in body copy (uses the brand / "cloud-
87     // storage" / passive
88     // voice) EXCEPT the technical ingest.uriAriaLabel + uriPlaceholder; #77 leaked them
89     // into kn/hi elsewhere.
90     describe("catalog content ? engine/server/bucket jargon only where en uses it (kn/hi)",
91     () => {
92         const ENGLISH = ["engine", "server", "bucket", "backend"] as const;
93         const TRANSLIT: Record<string, Record<(typeof ENGLISH)[number], string[]>> = {
94             kn: { engine: ["?????"], server: ["?????", "????????"], bucket: ["?????"], backend:
95             ["?????????"] },
96             hi: { engine: ["?????"], server: ["?????", "????????"], bucket: ["?????"], backend:

```

```

["?????"] },
91     };
92     for (const loc of ["kn", "hi"] as const) {
93         it(`${loc}: a jargon term (Latin or transliterated) appears only where en uses the
English word`, () => {
94             const e = load(loc);
95             const violations: Array<{ key: string; term: string; en: string }> = [];
96             for (const [key, val] of Object.entries(e)) {
97                 const enUsesWord = (w: string) => new RegExp(`\\b${w}\\b`, "i").test(en[key] ??
""));
98                 for (const w of ENGLISH) {
99                     const present = [
100                         ...(new RegExp(`\\b${w}\\b`, "i").test(val) ? [w] : []),
101                         ...TRANSLIT[loc][w].filter((t) => val.includes(t)),
102                     ];
103                     for (const term of present) if (!enUsesWord(w)) violations.push({ key, term,
en: en[key] ?? "(absent)" });
104                 }
105             }
106             expect(violations).toEqual([]);
107         });
108     }
109 });
110
111 // (4) Hindi de-Sanskritization (#77 class d) ? the over-Sanskritized / stiff terms must
stay replaced
112 // on the AUDITED keys. Key-scoped (not a repo-wide ban) so legitimate marketing prose
elsewhere is not
113 // constrained, while still catching a silent revert of the exact bug.
114 describe("catalog content ? Hindi de-Sanskritization stays fixed (#77 class d)", () => {
115     const hi = load("hi");
116     const BANNED: Array<[string, string]> = [
117         ["grid.shots", "?????????"], // "experimental" calque
118         ["grid.shotsTitle", "?????????"],
119         ["grid.shotsTitle", "?????????"], // "under development" bureaucratese
120         ["query.support", "?????????"],
121         ["query.support", "?????????"],
122         ["grid.shotsConfidence", "?????????"], // "faith" ? wrong sense for statistical
confidence
123         ["app.badge", "?????-????"], // "per-rally" calque
124     ];
125     for (const [key, term] of BANNED) {
126         it(`hi ${key} no longer contains the stiff "${term}"`, () => {
127             expect(hi).toHaveProperty(key);
128             expect(hi[key] ?? "").not.toContain(term);
129         });
130     }
131     it("hi keeps the friendlier replacements (??????? / ????? ?????)", () => {
132         expect(hi["grid.shots"]).toContain("?????????");
133         expect(hi["grid.shotsTitle"]).toContain("?????????");
134         expect(hi["query.support"]).toContain("?????????");
135         expect(hi["grid.shotsConfidence"]).toContain("????? ?????");
136     });
137 });
138
139 // (5) Flagged mistranslations ? substring guard (.not.toContain) so a padded/wrapped
variant of the
140 // banned term ("????? ?????????", a trailing space, ?) is still caught.
141 describe("catalog content ? flagged mistranslations stay fixed (kn/hi word choice)", ()
=> {
142     it("kn bulk.invert drops the 'rotate' mistranslation (????????)", () => {
143         expect(load("kn")["bulk.invert"]).not.toContain("????????");
144     });
145     it("kn result.dismiss drops the legalistic '????????'", () => {
146         expect(load("kn")["result.dismiss"]).not.toContain("????????");

```

```

147     });
148     it("hi result.dismiss drops the harsh '????'", () => {
149         expect(load("hi")["result.dismiss"]).not.toContain("????");
150     });
151 });
152

```

30. user (2026-06-22T03:19:14.773Z)

```

1      // CI guardrails that keep i18n "by design" (FRIENDLY_AND_MULTILINGUAL.md §4):
2      //   1. KEY-PARITY ? every locale catalog has EXACTLY the `en` key set (the
extensibility proof:
3      //       drop a stub locale and this lists every gap). Trivially green while only `en`
exists; it
4      //       starts enforcing the moment a kn/hi catalog lands (B5/B6).
5      //   2. NO-BARE-STRINGS (attribute level, v1) ? user-facing component attributes
(placeholder /
6      //       aria-label / title) must route through t(), never a hardcoded literal. A curated
start per
7      //       the doc; an AST-level sweep of JSX text can extend it later.
8      import { describe, it, expect } from "vitest";
9      import { readdirSync, readFileSync } from "node:fs";
10     import { join } from "node:path";
11
12     // Catalogs are hoisted to repo-root /locales (D-CATALOG-HOME): web/src/i18n ->
../../locales.
13     const LOCALES_DIR = join(import.meta.dirname, "..", "..", "..", "locales");
14     const SRC_DIR = join(import.meta.dirname, "..");
15
16     function flatten(obj: Record<string, unknown>, prefix = ""): string[] {
17         const keys: string[] = [];
18         for (const [k, v] of Object.entries(obj)) {
19             const key = prefix ? `${prefix}.${k}` : k;
20             if (v && typeof v === "object" && !Array.isArray(v)) keys.push(...flatten(v as
Record<string, unknown>, key));
21             else keys.push(key);
22         }
23         return keys.sort();
24     }
25
26     const loadKeys = (locale: string): string[] =>
27         flatten(JSON.parse(readFileSync(join(LOCALES_DIR, locale, "common.json"), "utf8")) as
Record<string, unknown>);
28
29     describe("i18n key-parity ? every locale has EXACTLY the en keys", () => {
30         const enKeys = loadKeys("en");
31         const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
32             .filter((d) => d.isDirectory() && d.name !== "en")
33             .map((d) => d.name);
34
35         it("en is the non-empty source catalog", () => {
36             expect(enKeys.length).toBeGreaterThan(20);
37         });
38
39         for (const loc of others) {
40             it(`${loc} matches en exactly (no missing, no extra keys)`, () => {
41                 const keys = loadKeys(loc);
42                 const missing = enKeys.filter((k) => !keys.includes(k));
43                 const extra = keys.filter((k) => !enKeys.includes(k));
44                 expect({ locale: loc, missing, extra }).toEqual({ locale: loc, missing: [], extra:
[] });
45             });
46         }
47     });
48

```

```

49     // Key-parity proves every locale has the same KEYS; this proves every locale's VALUE
keeps the same
50     // simple `{placeholder}` set as en ? so a translation can never silently drop a
{count}/{error}/{min}/
51     // {filename}/{n}/? and break interpolation. (ICU plural headers like `{count, plural,
?}` carry no
52     // simple {word} placeholder and match as empty on both sides, so they're correctly
ignored.)
53     describe("placeholder parity ? every locale preserves en's {placeholders}", () => {
54         function flatEntries(obj: Record<string, unknown>, prefix = ""): Record<string,
string> {
55             const out: Record<string, string> = {};
56             for (const [k, v] of Object.entries(obj)) {
57                 const key = prefix ? `${prefix}.${k}` : k;
58                 if (v && typeof v === "object" && !Array.isArray(v)) Object.assign(out,
flatEntries(v as Record<string, unknown>, key));
59                 else out[key] = String(v);
60             }
61             return out;
62         }
63         const placeholders = (s: string) => (s.match(/\{(\w+)\}/g) ?? []).sort();
64         const load = (loc: string) => flatEntries(JSON.parse(readFileSync(join(LOCALES_DIR,
loc, "common.json"), "utf8")));
65         const enE = load("en");
66         const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
67             .filter((d) => d.isDirectory() && d.name !== "en")
68             .map((d) => d.name);
69
70         for (const loc of others) {
71             it(`${loc} preserves every {placeholder} that en uses`, () => {
72                 const locE = load(loc);
73                 const mismatches: Array<{ key: string; en: string[]; got: string[] }> = [];
74                 for (const key of Object.keys(enE)) {
75                     if (!(key in locE)) continue; // missing keys are the key-parity test's job
76                     const a = placeholders(enE[key]);
77                     const b = placeholders(locE[key]);
78                     if (a.join(",") !== b.join(",")) mismatches.push({ key, en: a, got: b });
79                 }
80                 expect(mismatches).toEqual([]);
81             });
82         }
83     });
84
85     describe("es 3-form plural (CLDR one/many/other) ? MULTILINGUAL Phase A", () => {
86         // Spanish cardinal is {one, many, other}; every es ICU plural message MUST carry a
`many` arm
87         // (the `many` category fires on large/compact numbers) or intl-messageformat silently
falls back.
88         // These are the plural-bearing keys in the catalog.
89         const es = JSON.parse(readFileSync(join(LOCALES_DIR, "es", "common.json"), "utf8")) as
Record<string, Record<string, string>>;
90         const pluralKeys: Array<[string, string]> = [
91             ["footer", "count"],
92             ["footer", "floorSomeLead"],
93             ["selection", "count"],
94         ];
95         for (const [ns, key] of pluralKeys) {
96             it(`es ${ns}.${key} carries one/many/other arms`, () => {
97                 const msg = es[ns][key];
98                 expect(msg, `${ns}.${key}`).toMatch(/bone\s*\{/);
99                 expect(msg, `${ns}.${key}`).toMatch(/bmany\s*\{/);
100                 expect(msg, `${ns}.${key}`).toMatch(/bother\s*\{/);
101             });
102         }
103     });

```

```

104
105 describe("no-bare-strings ? user-facing component attributes go through t()", () => {
106     const FILES = [
107         "App.tsx",
108         "components/QueryBar.tsx",
109         "components/TapFilters.tsx",
110         "components/RallyGrid.tsx",
111         "components/SelectionFooter.tsx",
112         "components/CompileResult.tsx",
113         "components/LanguageSwitcher.tsx",
114         "components/IngestPanel.tsx",
115     ];
116     // A hardcoded placeholder / aria-label / title with letters is a bare-string
117     regression ? it
118     // must be an expression ({t(...)}), not a quoted literal.
119     const BARE_ATTR = /\b(?:placeholder|aria-label|title)\s*=\s*"^[^"]*[A-Za-z]{2}[^"]*"$/g;
120
121     for (const f of FILES) {
122         it(`${f}: no hardcoded placeholder/aria-label/title`, () => {
123             const src = readFileSync(join(SRC_DIR, f), "utf8");
124             expect(src.match(BARE_ATTR) ?? []).toEqual([]);
125         });
126     }
127
128 describe("Indic fonts (B4) ? self-hosted OFL Noto, no runtime CDN", () => {
129     it("main.tsx bundles the OFL Noto Kannada + Devanagari fonts (self-hosted, not Google
130     Fonts)", () => {
131         const main = readFileSync(join(SRC_DIR, "main.tsx"), "utf8");
132         expect(main).toMatch(/@fontsource-variable\/noto-sans-kannada/);
133         expect(main).toMatch(/@fontsource-variable\/noto-sans-devanagari/);
134     });
135     it("the font stack falls back to Noto for Kannada + Devanagari glyphs", () => {
136         const css = readFileSync(join(SRC_DIR, "index.css"), "utf8");
137         expect(css).toMatch(/Noto Sans Kannada Variable/);
138         expect(css).toMatch(/Noto Sans Devanagari Variable/);
139     });
140
141 // The Studio SPA must render correctly on an AIR-GAPPED appliance (GPU/CPU/RAM, no
142 internet) ?
143 // every asset (fonts, styles, scripts) is bundled local, nothing pulled from a
144 CDN/Google Fonts.
145 describe("offline / self-contained ? no external runtime assets", () => {
146     const CDN = /googleapis|gstatic|fonts\.google|cdnjs|unpkg|jsdelivr\bcdn\.[a-z]/i;
147     const FILES = [
148         "../index.html", // web/index.html
149         "main.tsx",
150         "index.css",
151     ];
152     for (const rel of FILES) {
153         it(`${rel}: no external font/CDN reference`, () => {
154             expect(readFileSync(join(SRC_DIR, rel), "utf8")).not.toMatch(CDN);
155         });
156     }
157     it("web/index.html loads no absolute-URL <link>/<script src> (everything is
158     bundled/local)", () => {
159         const html = readFileSync(join(SRC_DIR, "..", "index.html"), "utf8");
160         expect(html).not.toMatch(/(?:href|src)\s*=\s*"["']https?:\/\//i);
161     });
162 }

```

Perfect! Now let me find the locales directory and check one catalog, then look for the test setup and the bundle_ui script:

32. user (2026-06-22T03:19:18.657Z)

```
C:\Users\avidu\Projects\khelsutra/locales
C:\Users\avidu\Projects\khelsutra/site/public/locales
```

33. user (2026-06-22T03:19:18.906Z)

```
1      // Vitest setup (runs before each test file's imports). DOM-env only: jsdom under Node
26 does not
2      // expose a Web Storage `localStorage`, which the i18n language detector and the
persistence test
3      // both need. Node-env test files (typeof window === "undefined") skip this entirely.
4      if (typeof window !== "undefined" && typeof globalThis.localStorage === "undefined") {
5          let store: Record<string, string> = {};
6          const mem: Storage = {
7              get length() {
8                  return Object.keys(store).length;
9              },
10             clear() {
11                 store = {};
12             },
13             getItem(key: string) {
14                 return Object.prototype.hasOwnProperty.call(store, key) ? store[key] : null;
15             },
16             key(index: number) {
17                 return Object.keys(store)[index] ?? null;
18             },
19             removeItem(key: string) {
20                 delete store[key];
21             },
22             setItem(key: string, value: string) {
23                 store[key] = String(value);
24             },
25         };
26         Object.defineProperty(window, "localStorage", { value: mem, configurable: true,
writable: true });
27         Object.defineProperty(globalThis, "localStorage", { value: mem, configurable: true,
writable: true });
28     }
29
```

34. user (2026-06-22T03:19:21.286Z)

```
1      {
2          "app": {
3              "badge": "pick your rallies · alpha",
4              "title": "Pick the rallies for your reel",
5              "intro": "Filter by <b>rally length</b> ? e.g. <i>?top 15 longest?</i> ? then fine-
6                  tune by tapping cards. Everything starts <b>selected</b>; just drop the ones you don't want."
7          },
8          "lang": {
9              "label": "Language",
10             "en": "English",
11             "kn": "?????",
12             "hi": "??????",
13             "es": "Español",
14             "da": "Dansk",
15             "id": "Bahasa Indonesia"
16         },
17         "bulk": {
18             "selectAll": "Select all",
19         }
20     }
```



```

18     "clear": "Clear",
19     "invert": "Invert"
20 },
21 "mode": {
22     "demo": "These are sample rallies ? process a video above to work on a real match.",
23     "loading": "Finding the rallies in your video?",
24     "error": "Couldn't load this video's rallies: {error}",
25     "loaded": "Found <b>{count}</b> rallies in your video."
26 },
27 "query": {
28     "placeholder": "e.g. over 10s · top 15 longest · shortest first",
29     "ariaLabel": "Filter rallies by length, count or order",
30     "apply": "Apply",
31     "notAvailable": "not available yet",
32     "notAvailableTitle": "?{token}? needs {kind} detection ? not available yet",
33     "notAvailableHelp": "Shot type, players and score aren't available yet ? filter by
rally length instead.",
34     "unreadable": "Couldn't read that ? try ?over 10s? or ?top 15 longest?.",
35     "hint": "Type over 10s for rallies longer than 10 seconds ? or tap a sample below.",
36     "try": "Sample queries:",
37     "support": "Filtering by <b>rally length</b> is fully supported. Shot-count is
<x>experimental</x> (under development).",
38 },
39 "grid": {
40     "empty": "No rallies to show yet.",
41     "rally": "Rally {n}",
42     "inReel": "Rally {n}, in reel",
43     "notInReel": "Rally {n}, not in reel",
44     "start": "start",
45     "long": "long",
46     "shots": "~{count} shots · experimental",
47     "shotsTitle": "Shot count is experimental (under development) and may be
inaccurate{conf}. Filter by rally length instead.",
48     "shotsConfidence": " ? confidence: {conf}"
49 },
50 "footer": {
51     "count": "{count, plural, one {# rally} other {# rallies}}",
52     "summaryRest": "selected · {total} total",
53     "reelName": "Reel name",
54     "reelNamePlaceholder": "my-highlights",
55     "build": "Build reel ({count})",
56     "building": "Building?",
57     "floorOne": "<b>Your only selected rally</b> has fewer than {min} shots ? it would
be skipped, so the build would fail. Pick a longer rally.",
58     "floorAll": "<b>All {count} selected rallies</b> have fewer than {min} shots ?
they'd all be skipped, so the build would fail. Pick longer rallies.",
59     "floorSomeLead": "{count, plural, one {# rally} other {# rallies}}",
60     "floorSomeRest": "with fewer than {min} shots will be skipped when building (they're
too short).",
61 },
62 "result": {
63     "share": "Share on WhatsApp",
64     "shareText": "I made my badminton highlights reel with KhelSutra ? Try it:
khelsutra.guru",
65     "ready": "Your reel is ready",
66     "failed": "Build failed",
67     "building": "Stitching your reel?",
68     "buildingReassure": "Joining your selected rallies into one video ? this usually
only takes a moment.",
69     "dismiss": "Dismiss",
70     "download": "Download {filename}"
71 },
72 "build": {
73     "noVideo": "These are sample rallies ? process a real video first to build a reel."
74 },

```

```

75     "ingest": {
76         "title": "Process a video",
77         "intro": "Point KhelSutra at a video and it finds the rallies for you ? this can
take a few minutes.",
78         "uriPlaceholder": "gs://your-bucket/match.mp4",
79         "uriAriaLabel": "Video location ? a gs:// bucket URL or a path on the engine host",
80         "hint": "Paste a cloud-storage link (gs://?), or a path to a video on the machine
running KhelSutra.",
81         "submit": "Process video",
82         "submitting": "Submitting?",
83         "cancel": "Cancel",
84         "working": "Processing your video?",
85         "progress": "{progress}",
86         "retry": "Try again",
87         "reassure": "Safe to wait ? KhelSutra keeps working even if your connection is slow;
a long video can take several minutes.",
88         "statusFailed": "Couldn't process",
89         "chooseFile": "Choose a video file",
90         "uploadAria": "Choose a video file to upload",
91         "fileHint": "Pick an .mp4 or .mov from this device ? it uploads, then KhelSutra
finds the rallies.",
92         "orLink": "Or use a cloud link",
93         "uploadingStatus": "Uploading?",
94         "uploadProgress": "Upload progress"
95     },
96     "errors": {
97         "networkError": "Can't reach KhelSutra. Is it running?",
98         "invalidUri": "Enter a video location first ? a gs:// link or a path on the machine
running KhelSutra.",
99         "emptyResult": "Processing finished but found no rallies in this video. Try a
clearer match recording.",
100        "timeout": "This is taking longer than expected. The job may still be running ?
reload with the video link, or try again.",

```

35. user (2026-06-22T03:19:21.902Z)

Exit code 2

/usr/bin/bash: eval: line 1: unexpected EOF while looking for matching `''

36. user (2026-06-22T03:19:24.324Z)

```

locales\da\common.json
locales\en\common.json
locales\es\common.json
locales\hi\common.json
locales\id\common.json
locales\kn\common.json

```

37. user (2026-06-22T03:19:25.317Z)

C:\Users\avidu\Projects\bhi-p2b/scripts/bundle_ui.sh

38. user (2026-06-22T03:19:27.562Z)

```

1     #!/usr/bin/env bash
2     # Bundle the KhelSutra web SPA into the engine so a downloaded engine serves the real UI
3     # single-origin (PACKAGING_PLAN.md P1b).
4     #
5     # OPTION 1 (now, owner-chosen): build the SPA from a khelsutra checkout and vendor the
built `dist/`
6     # into `backend/ui/`, which is COMMITTED ? so the wheel + CI build fully
offline/reproducibly. Refresh
7     # the snapshot by re-running this on an SPA release; commit the diff.
8     #

```

```

9      # OPTION 3 (designed-for, future): point --from at a published dist tarball/URL (a
GitHub Release
10     # artifact) to vendor a pre-built SPA without a local khelsutra checkout or Node.
Already wired below.
11     #
12     # Usage:
13     #   scripts/bundle_ui.sh [SOURCE]
14     #   SOURCE = a khelsutra checkout dir (default: $KHELSUTRA_DIR or ../khelsutra)
[builds it]
15     #           | a path/URL ending .tgz/.tar.gz
[unpacks it]
16     #   BUILD=0 scripts/bundle_ui.sh <khelsutra-dir> # skip npm; reuse an existing
web/dist
17     set -euo pipefail
18
19     ENGINE_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
20     UI_DEST="$ENGINE_ROOT/backend/ui"
21     SOURCE="${1:-${KHELSUTRA_DIR:-$ENGINE_ROOT/../../khelsutra}}"
22
23     stage_dist() { # populates $1 (a dist dir) into a clean $UI_DEST
24         local dist="$1"
25         [ -f "$dist/index.html" ] || { echo "ERROR: no index.html in $dist ? not a built SPA
dist" >&2; exit 1; }
26         rm -rf "$UI_DEST"
27         mkdir -p "$UI_DEST"
28         cp -r "$dist"/. "$UI_DEST"/
29     }
30
31     SPA_REPO="unknown"
32     SPA_COMMIT="unknown"
33
34     case "$SOURCE" in
35         *.tgz|*.tar.gz|http://*|https://*)
36             # OPTION 3: a published dist artifact.
37             tmp="$(mktemp -d)"; trap 'rm -rf "$tmp"' EXIT
38             if [ [ "$SOURCE" == http*://* ] ]; then
39                 echo "[bundle_ui] fetching dist artifact: $SOURCE"
40                 curl -fsSL "$SOURCE" -o "$tmp/dist.tgz"; SRC_TGZ="$tmp/dist.tgz";
SPA_REPO="$SOURCE"
41             else
42                 SRC_TGZ="$SOURCE"; SPA_REPO="$SOURCE"
43             fi
44             tar -xzf "$SRC_TGZ" -C "$tmp"
45             # accept either dist/* at the root or a top-level dir containing index.html
46             if [ -f "$tmp/index.html" ]; then DIST="$tmp"; else DIST="$(dirname "$(find "$tmp"
-name index.html | head -1) )"; fi
47             stage_dist "$DIST"
48             ;;
49         *)
50             # OPTION 1: a khelsutra checkout ? build it.
51             WEB="$SOURCE/web"
52             [ -d "$WEB" ] || { echo "ERROR: $WEB not found (pass a khelsutra checkout dir or a
dist tarball)" >&2; exit 1; }
53             SPA_REPO="$(git -C "$SOURCE" config --get remote.origin.url 2>/dev/null || echo
"$SOURCE")"
54             SPA_COMMIT="$(git -C "$SOURCE" rev-parse HEAD 2>/dev/null || echo unknown)"
55             if [ "${BUILD:-1}" = "1" ]; then
56                 echo "[bundle_ui] building SPA in $WEB (npm ci && npm run build)"
57                 ( cd "$WEB" && npm ci && npm run build )
58             else
59                 echo "[bundle_ui] BUILD=0 ? reusing existing $WEB/dist"
60             fi
61             stage_dist "$WEB/dist"
62             ;;
63     esac

```

```

64
65     # Third-party notices for the vendored bundle (the python license-scan lane can't see
JS/font blobs).
66     cat > "$UI_DEST/THIRD_PARTY_NOTICES.md" <<NOTICE
67     # Third-party notices ? vendored KhelSutra SPA (`backend/ui/\`)
68
69     This directory is the BUILT KhelSutra web SPA (MIT), vendored from $SPA_REPO @
$SPA_COMMIT by
70     `scripts/bundle_ui.sh` (PACKAGING_PLAN.md P1b). The minified `assets/*.js` bundle
includes
71     third-party runtime dependencies (React and the SPA's other libraries ? MIT/BSD), and
the
72     `assets/*.woff2` are Noto Sans Kannada/Devanagari under the SIL Open Font License 1.1
(decision D7).
73     Per the project's MIT/Apache/BSD-only guardrail, the SPA's dependencies are vetted in
the SPA repo;
74     see its package manifest/lockfile for the authoritative per-package licenses.
75     NOTICE
76
77     # Provenance + the SPA<->engine contract version (the engine's API_VERSION at bundle
time). The
78     # content hash is over the SPA build output only (relative paths ? reproducible across
checkouts),
79     # excluding the two generated metadata files.
80     ENGINE_API_VERSION="$(grep -E '^API_VERSION\b[^\s]*=' "$ENGINE_ROOT/backend/main.py" |
grep -oE '[0-9]+' | head -1)"
81     CONTENT_HASH="$( (cd "$UI_DEST" && find . -type f ! -name ui_version.json ! -name
THIRD_PARTY_NOTICES.md | LC_ALL=C sort | xargs sha256sum) | sha256sum | cut -d' ' -f1)"
82     BUILT_AT="$(date -u +%Y-%m-%dT%H:%M:%SZ 2>/dev/null || echo unknown)"
83
84     cat > "$UI_DEST/ui_version.json" <<JSON
85     {
86     "spa_repo": "$SPA_REPO",
87     "spa_commit": "$SPA_COMMIT",
88     "engine_api_version": ${ENGINE_API_VERSION:-null},
89     "content_hash": "$CONTENT_HASH",
90     "built_at": "$BUILT_AT"
91     }
92     JSON
93
94     echo "[bundle_ui] vendored SPA -> $UI_DEST (spa_commit=$SPA_COMMIT,
engine_api_version=$ENGINE_API_VERSION)"
95     echo "[bundle_ui] files:"; find "$UI_DEST" -type f | sed "s|$UI_DEST/||" | sort
96

```

39. user (2026-06-22T03:19:28.419Z)

```

C:\Users\avidu\Projects\kheIsutra\web\src\lib\errors.ts
C:\Users\avidu\Projects\kheIsutra\web\src\lib\errorText.ts

```

40. user (2026-06-22T03:19:30.735Z)

```

1      // Error helpers shared by the ingest flow and the build flow. Kept i18n-FREE: callers
map an
2      // IngestErrorCode to a localized string (so hooks stay testable without a translation
context),
3      // while the engine's own `detail` text passes through verbatim (it's the most specific
message).
4      import { ApiError } from "../api/client";
5
6      /** Pull FastAPI's {"detail": "..."} message out of an error body, else return the raw
body. */
7      export function parseDetail(body: string): string {
8          try {
9              const j = JSON.parse(body) as { detail?: unknown };

```

```

10         if (j && typeof j.detail === "string") return j.detail;
11     } catch {
12         /* not JSON ? fall through to the raw body */
13     }
14     return body;
15 }
16
17 /** Stable, localizable failure categories surfaced by the ingest flow.
18  * `engine` carries the engine's own message in `detail`; the rest are localized by
19  code. */
20
21 export type IngestErrorCode = "invalidUri" | "networkError" | "emptyResult" | "timeout"
22 | "engine";
23
24 export interface IngestError {
25     code: IngestErrorCode;
26     /** Present for `engine` ? the verbatim FastAPI detail / status; rendered as-is, not
27     translated. */
28     detail?: string;
29 }
30
31 /** Classify a thrown client error into a stable code. A network throw (ApiError with no
32 `status`)
33  * is the common "engine unreachable" case ? its own friendly code; any HTTP error
34  surfaces the
35  * engine's `detail` (e.g. an unsupported-scheme 400, a not-found 404). */
36 export function classifyError(err: unknown): IngestError {
37     if (err instanceof ApiError) {
38         if (err.status === undefined) return { code: "networkError" }; // fetch threw ?
39         engine truly unreachable
40         if (err.body) {
41             const detail = parseDetail(err.body);
42             if (detail) return { code: "engine", detail }; // a real engine message (e.g. a
43             400 unsupported-scheme)
44         }
45         // A bare 5xx with no body is almost always the engine being down / a gateway that
46         can't reach it
47         // (e.g. the dev proxy when :8000 is off) ? for a non-technical user a friendly "is
48         it running?"
49         // beats a cryptic "HTTP 500". A 4xx with no body keeps the status (it's a client-
50         side mistake).
51         if (err.status >= 500) return { code: "networkError" };
52         return { code: "engine", detail: `HTTP ${err.status}` };
53     }
54     return { code: "engine", detail: String(err) };
55 }
56
57
58
59

```

41. user (2026-06-22T03:19:31.068Z)

```

1     // The ONE place a classified error (lib/errors.ts) becomes user-facing text. Both
2     surfaces use it:
3     // the ingest flow (IngestPanel) and the build flow (App.onBuild ? CompileResult), so a
4     5xx / engine-
5     // down failure reads the same friendly way everywhere instead of a raw "HTTP 500" /
6     fetch message.
7     //
8     // `engine` carries the engine's OWN message (the most specific) ? shown verbatim, never
9     translated.
10    // Every other code maps to a friendly localized line in the shared `errors.*`
11    namespace. Kept out of
12    // errors.ts so that module stays i18n-free and unit-testable without a translation
13    context.
14
15    import type { TFunction } from "i18next";
16    import type { IngestError } from "../errors";
17
18
19

```

```

11     export function errorText(t: TFunction, e: IngestError): string {
12         if (e.code === "engine") return e.detail ?? t("errors.engineGeneric");
13         return t(`errors.${e.code}`);
14     }
15

```

42. assistant (2026-06-22T03:19:33.066Z)

Now let me check how localStorage is used and look for any existing first-run patterns:

43. user (2026-06-22T03:19:34.588Z)

```

C:\Users\avidu\Projects\khelsutra\web\src\components\A4.Touch.Test.tsx:20:
window.localStorage.clear();
C:\Users\avidu\Projects\khelsutra\web\src\components\A8.Status.Test.tsx:13:
window.localStorage.clear();
C:\Users\avidu\Projects\khelsutra\web\src\components\Build-Error.Test.tsx:39:
window.localStorage.clear();
C:\Users\avidu\Projects\khelsutra\web\src\components\IngestPanel.Test.tsx:33:
window.localStorage.clear();
C:\Users\avidu\Projects\khelsutra\web\src\components\LanguageSwitcher.tsx:17: * detector caches
to localStorage (LANG_STORAGE_KEY) ? so the choice is the top-priority signal on
C:\Users\avidu\Projects\khelsutra\web\src\components\QueryBar.Test.tsx:13:
window.localStorage.clear();
C:\Users\avidu\Projects\khelsutra\web\src\components\SelectionFooter.Test.tsx:25:
window.localStorage.clear();
C:\Users\avidu\Projects\khelsutra\web\src\components\SelectionFooter.Test.tsx:57: beforeEach(()
=> window.localStorage.clear());
C:\Users\avidu\Projects\khelsutra\web\src\components\SelectionFooter.Test.tsx:89:
window.localStorage.clear();
C:\Users\avidu\Projects\khelsutra\web\src\components\Upload.Test.tsx:51:
window.localStorage.clear();
C:\Users\avidu\Projects\khelsutra\web\src\i18n\i18n.Test.tsx:15: window.localStorage.clear();
C:\Users\avidu\Projects\khelsutra\web\src\i18n\i18n.Test.tsx:47:
expect(window.localStorage.getItem(LANG_STORAGE_KEY)).toBe("kn");
C:\Users\avidu\Projects\khelsutra\web\src\i18n\i18n.Test.tsx:92: window.localStorage.clear();
// no persisted preference yet
C:\Users\avidu\Projects\khelsutra\web\src\i18n\i18n.Test.tsx:98: window.localStorage.clear();
C:\Users\avidu\Projects\khelsutra\web\src\i18n\i18n.Test.tsx:104:
window.localStorage.setItem(LANG_STORAGE_KEY, "kn");
C:\Users\avidu\Projects\khelsutra\web\src\i18n\i18n.Test.tsx:111:
expect(order.indexOf("localStorage")).toBeGreaterThanOrEqual(0);
C:\Users\avidu\Projects\khelsutra\web\src\i18n\i18n.Test.tsx:112:
expect(order.indexOf("localStorage")).toBeLessThan(order.indexOf("navigator"));
C:\Users\avidu\Projects\khelsutra\web\src\i18n\i18n.Test.tsx:117:
window.localStorage.clear();
C:\Users\avidu\Projects\khelsutra\web\src\i18n\i18n.Test.tsx:165:
window.localStorage.clear();
C:\Users\avidu\Projects\khelsutra\web\src\i18n\index.ts:60: order: ["localStorage",
"querystring", "navigator"],
C:\Users\avidu\Projects\khelsutra\web\src\i18n\index.ts:63: caches: ["localStorage"],
C:\Users\avidu\Projects\khelsutra\web\src\test\setup.ts:2:// expose a Web Storage
`localStorage`, which the i18n language detector and the persistence test
C:\Users\avidu\Projects\khelsutra\web\src\test\setup.ts:4:if (typeof window !== "undefined" &&
typeof globalThis.localStorage === "undefined") {
C:\Users\avidu\Projects\khelsutra\web\src\test\setup.ts:26: Object.defineProperty(window,
"localStorage", { value: mem, configurable: true, writable: true });
C:\Users\avidu\Projects\khelsutra\web\src\test\setup.ts:27: Object.defineProperty(globalThis,
"localStorage", { value: mem, configurable: true, writable: true });

```

44. user (2026-06-22T03:19:35.275Z)

No files found

45. assistant (2026-06-22T03:19:37.279Z)

Let me check the LanguageSwitcher component for the localStorage pattern:

46. user (2026-06-22T03:19:38.037Z)

```
1      import { useTranslation } from "react-i18next";
2      import { SUPPORTED_LOCALES, DEFAULT_LOCALE, type Locale } from "../i18n";
3
4      // Each locale's label is its own autonym, shown in its own script. Kannada renders
crisply once the
5      // bundled Noto Sans Kannada font lands (tracker B4); most devices already ship a system
Kannada font.
6      const LANG_LABEL_KEY: Record<Locale, string> = {
7          en: "lang.en",
8          kn: "lang.kn",
9          hi: "lang.hi",
10         es: "lang.es",
11         da: "lang.da",
12         id: "lang.id",
13     };
14
15     /**
16      * A persistent language switcher. Selecting a locale calls i18next's changeLanguage,
which the
17      * detector caches to localStorage (LANG_STORAGE_KEY) ? so the choice is the top-
priority signal on
18      * the next visit (negotiation: persisted pref wins). Icon-free + non-color active state
via
19      * aria-pressed keeps it usable for non-tech-savvy users (FRIENDLY_AND_MULTILINGUAL §4).
20      */
21     export function LanguageSwitcher() {
22         const { t, i18n } = useTranslation();
23         // Highlight the user's CHOSEN locale (i18n.language), not the resolved-for-content
one:
24         // resolvedLanguage falls back to `en` when a catalog is missing, so picking Kannada
(content
25         // still English until the kn catalog lands, B5) must still show Kannada as selected.
Compare on
26         // the base subtag so a detected "en-IN" highlights "en".
27         const active = (i18n.language || DEFAULT_LOCALE).split("-")[0];
28
29         return (
30             <nav aria-label={t("lang.label")} className="flex items-center gap-1">
31                 {SUPPORTED_LOCALES.map((loc) => {
32                     const isActive = active === loc;
33                     return (
34                         <button
35                             key={loc}
36                             type="button"
37                             lang={loc}
38                             aria-pressed={isActive}
39                             onClick={() => void i18n.changeLanguage(loc)}
40                             className={[
41                                 "inline-flex min-h-11 items-center rounded-full px-4 text-sm font-semibold
transition",
42                                 isActive ? "bg-white text-ink" : "text-white/70 hover:text-white",
43                                 ].join(" ")}
44                             >
45                             {t(LANG_LABEL_KEY[loc])}
46                         </button>
47                     );
48                 })}
49             </nav>
50         );
```

```
51     }
52
```

47. user (2026-06-22T03:19:38.892Z)

```
12:   "test": "vitest run",
13:   "test:coverage": "vitest run --coverage"
27:   "@testing-library/dom": "^10.4.1",
28:   "@testing-library/react": "^16.3.2",
33:   "@vitest/coverage-v8": "^3.2.6",
38:   "vitest": "^3.0.0"
```

48. assistant (2026-06-22T03:19:41.371Z)

Perfect! Now let me create a comprehensive map. Let me also check the main.tsx and vitest config to complete the picture:

49. user (2026-06-22T03:19:42.074Z)

```
1     import { StrictMode } from "react";
2     import { createRoot } from "react-dom/client";
3     import App from "./App";
4     import "./i18n"; // initialize the i18next singleton before the app renders (tracker B1)
5     // Self-hosted OFL Noto fonts so Kannada/Hindi render crisply offline (tracker B4 /
decision D7) ?
6     // no Google Fonts runtime dependency. unicode-range-scoped, so only Indic codepoints
use them.
7     import "@fontsource-variable/noto-sans-kannada/index.css";
8     import "@fontsource-variable/noto-sans-devanagari/index.css";
9     import "./index.css";
10
11     const root = document.getElementById("root");
12     if (!root) throw new Error("#root not found");
13
14     createRoot(root).render(
15       <StrictMode>
16         <App />
17       </StrictMode>,
18     );
19
```

50. user (2026-06-22T03:19:43.117Z)

C:\Users\avidu\Projects\kheIsutra\web\node_modules/i18next-icu\tsconfig.json
C:\Users\avidu\Projects\kheIsutra\web\tsconfig.json

51. user (2026-06-22T03:19:46.223Z)

(Bash completed with no output)

52. user (2026-06-22T03:19:46.574Z)

```
1     {
2       "compilerOptions": {
3         "target": "ES2022",
4         "useDefineForClassFields": true,
5         "lib": ["ES2022", "DOM", "DOM.Iterable"],
6         "module": "ESNext",
7         "skipLibCheck": true,
8
9         "moduleResolution": "bundler",
10        "allowImportingTsExtensions": true,
11        "resolveJsonModule": true,
12        "isolatedModules": true,
```



```

13     "moduleDetection": "force",
14     "noEmit": true,
15     "jsx": "react-jsx",
16
17     "strict": true,
18     "noUnusedLocals": true,
19     "noUnusedParameters": true,
20     "noFallthroughCasesInSwitch": true,
21
22     "types": ["vite/client", "node"]
23 },
24 "include": ["src", "vite.config.ts"]
25 }
26

```

53. user (2026-06-22T03:19:48.776Z)

```

1  import { defineConfig } from "vitest/config";
2  import react from "@vitejs/plugin-react";
3  import tailwindcss from "@tailwindcss/vite";
4
5  // The SPA makes SAME-ORIGIN relative `/api/*` calls. In the appliance (prod) a reverse
6  // proxy serves the SPA and proxies `/api` to the engine on 127.0.0.1:8000
7  // (docs/LOCAL_APPLIANCE_ARCHITECTURE.md). In dev, Vite proxies `/api` to the local
engine
8  // (`python -m backend.main --server`, :8000). Override with VITE_ENGINE_ORIGIN if
needed.
9  export default defineConfig({
10    plugins: [react(), tailwindcss()],
11    server: {
12      proxy: {
13        "/api": { target: "http://localhost:8000", changeOrigin: true },
14      },
15    },
16    test: {
17      // Default env is node (the pure lib/parser tests); the i18n render test opts into
jsdom via a
18      // `@vitest-environment jsdom` docblock. Match .test.tsx too so render tests are
discovered.
19      environment: "node",
20      include: ["src/**/*.test.{ts,tsx}"],
21      // DOM-env localStorage polyfill (jsdom under Node 26 ships none); node-env files
no-op.
22      setupFiles: ["src/test/setup.ts"],
23      // Coverage floor (CI runs `npm run test:coverage`): a merge can't DROP coverage
below these
24      // thresholds, so untested new code is caught. `all` counts every source file (not
only the ones a
25      // test imported), so deleting a test surfaces here too. Excludes are non-logic
files: the app
26      // entry/bootstrap, type-only declarations, and the test harness itself.
27      coverage: {
28        provider: "v8",
29        reporter: ["text", "text-summary"],
30        all: true,
31        include: ["src/**/*.test.{ts,tsx}"],
32        exclude: [
33          "src/**/*.test.{ts,tsx}", // the tests
34          "src/test/**", // jsdom localStorage shim (test harness, not shipped logic)
35          "src/main.tsx", // app entry: mounts <App/> + bundles fonts ? nothing to unit-
test
36          "src/api/types.ts", // type-only declarations (no runtime code)
37          "**/*.d.ts",
38        ],
39      // Floor set just below current (stmts/lines 93.3, branches 88.1, funcs 85.9) ? a

```

```

real ratchet
40          // that fails CI on a meaningful drop, with a little slack so it doesn't flap.
Ratchet upward as
41          // coverage grows. (functions stays a touch looser: App.tsx is the engine-
orchestrating shell,
42          // costly to exercise fully in unit tests.)
43          thresholds: {
44              statements: 92,
45              branches: 87,
46              functions: 84,
47              lines: 92,
48          },
49      },
50  },
51  });
52

```

54. assistant (2026-06-22T03:21:00.391Z)

Excellent! Now I have everything I need. Let me compile the comprehensive map:

Tight Structured Map: Khelsutra Web SPA for P2C First-Run Wizard

1. **App Mount & Screen State**

File: `web/src/App.tsx` **Lines:** 49?246

State that decides which screen to show:

```

` ` `tsx
// Line 54?56: videoId is the KEY signal
const [videoId, setVideoId] = useState<string | null>()
  () => new URLSearchParams(window.location.search).get("video"),
);
` ` `

```

Conditional render: Lines 239?246

```

` ` `tsx
{ /* Front-half ingest (B-P2): when no video is loaded, point the tool at a bucket URI / host
  path ? process ? load. Once a video_id exists, the existing back-half takes over. */ }
{ !videoId && (
  <IngestPanel
    onLoaded={(id) => {
      setVideoId(id);
      window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
    }}
  />
)}
` ` `

```

First-run signals:

- `videoId === null` ? IngestPanel shown (front-half ingest, no processing yet)
- `loadState.status === "demo"` ? DEMO_RALLIES shown (default state when videoId is null)
- **Hook point:** A `before the IngestPanel (lines 228?246), conditional on a "wizard.completed" localStorage flag and `videoId === null`. The wizard's final step hands the first Gemini key + compute config to the engine, then calls `onLoaded(demoVideoId)` to unblock the UI.

2. **API Client Pattern**

Files: `web/src/api/client.ts` **Lines:** 1?79 (request fn), 82?134 (public endpoints)

BASE URL logic: Lines 23?27

```

```tsx
export function apiBase(): string {
 return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
}
const BASE = apiBase();
```

**How to add an endpoint ? template from `getConfig()`:** Lines 131?134
```tsx
export async function getConfig(cid?: string): Promise<EngineConfig> {
 return request<EngineConfig>("/config", { method: "GET", cid });
}
```

**For `/api/capabilities`:**
```tsx
export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
 return request<EngineCapabilities>("/capabilities", { method: "GET", cid });
}
```

**Request idiom:** Lines 51?79
- Calls `fetch(url, { method, headers: { "x-correlation-id": cid, ... } })`
- On `!res.ok`: throws `ApiError` with `{ status, body }`
- On network throw: `ApiError` with `cid` only (status=undefined)

**Error handling:** Lines 30?41 (`ApiError` class)
```tsx
export class ApiError extends Error {
 readonly cid?: string;
 readonly status?: number;
 readonly body?: string;
}
```

**Existing `/api/config` call:** Line 106 in `App.tsx`, used to read
`cfg.stitching.min_shot_count`

---

```

3. **`/api/capabilities` Shape ? Wizard Steps**

```

**Engine returns:**
```json
{
 "api_version": "...",
 "engine_version": "...",
 "segmenters": [...],
 "default_segmenter": "...",
 "ffmpeg": { "ffmpeg": bool, "ffprobe": bool, "ffmpeg_path": "...", "ffprobe_path": "..." },
 "gpu": { "nvidia_smi_present": bool },
 "gemini_key_present": bool,
 "wasb_weights_present": bool,
 "deployment": { "profile": "...", "compute_target": "..." }
}
```

**Wizard's 3 steps use:**
1. Gemini Key Step: `gemini_key_present` ? if false, show key-entry/guided-setup
2. Compute Step: `deployment.profile`, `deployment.compute_target` ? show which compute is active
3. Videos Step: `ffmpeg.ffmpeg`, `ffmpeg.ffprobe`, `gpu.nvidia_smi_present` ? show available ingestion methods

**Type to add to `web/src/api/types.ts`:

```

```

```tsx
export interface EngineCapabilities {
 api_version: string;
 engine_version: string;
 segmenters?: string[];
 default_segmenter?: string;
 ffmpeg?: {
 ffmpeg: boolean;
 ffprobe: boolean;
 ffmpeg_path?: string;
 ffprobe_path?: string;
 };
 gpu?: { nvidia_smi_present: boolean };
 gemini_key_present: boolean;
 wasb_weights_present?: boolean;
 deployment?: { profile?: string; compute_target?: string };
 [key: string]: unknown;
}
```

```

4. ****i18n Catalogs + Parity Guard****

```

**Catalog locations:** `~/locales/{en,kn,hi,es,da,id}/common.json`
**Import in web:** `web/src/i18n/index.ts` **Lines:** 16?21
```tsx
import enCommon from "../../locales/en/common.json";
import knCommon from "../../locales/kn/common.json";
import hiCommon from "../../locales/hi/common.json";
import esCommon from "../../locales/es/common.json";
import daCommon from "../../locales/da/common.json";
import idCommon from "../../locales/id/common.json";
```

**SUPPORTED_LOCALES:** Line 25
```tsx
export const SUPPORTED_LOCALES = ["en", "kn", "hi", "es", "da", "id"] as const;
```

**KEY-PARITY Guard:** `web/src/i18n/hygiene.test.ts` **Lines:** 29?46
```tsx
describe("i18n key-parity ? every locale has EXACTLY the en keys", () => {
 const enKeys = loadKeys("en");
 const others = readdirSync(LOCALES_DIR, { withFileTypes: true })
 .filter((d) => d.isDirectory() && d.name !== "en")
 .map((d) => d.name);

 for (const loc of others) {
 it(`${loc} matches en exactly (no missing, no extra keys)`, () => {
 const keys = loadKeys(loc);
 const missing = enKeys.filter((k) => !keys.includes(k));
 const extra = keys.filter((k) => !enKeys.includes(k));
 expect({ locale: loc, missing, extra }).toEqual({ locale: loc, missing: [], extra: [] });
 });
 }
});
```

```

****PLACEHOLDER-PARITY Guard:**** Lines 53?83 ? enforces `{count}`, `{error}`, etc. match across locales

****CRITICAL:**** You MUST add wizard keys (e.g., `wizard.step1.title`, `wizard.step2.description`) to ****ALL 6 catalogs**** with matching keys and placeholder sets, or CI fails on the parity test. The es catalog must use ICU 3-form plurals for plural keys (lines 85?103).

```

**Catalog structure:** Nested namespaces, flat rendering via dot notation
``json
{
  "wizard": {
    "step1": { "title": "Add Gemini Key", ... },
    "step2": { ... },
    ...
  }
}
...

**t() usage in a component:**
`IngestPanel.tsx` **Lines:** 56, 69, 79, 100, etc.
``tsx
const { t, i18n } = useTranslation(); // hook at component top
// Interpolation:
t("ingest.progress", { progress: phase.progress }) // Line 147
// With markup (Trans):
<Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} /> // App.tsx Line 234
...

---
```

5. ****Component + Test Conventions****

****Template:** IngestPanel.tsx + IngestPanel.test.tsx******

```

**Component structure** (`IngestPanel.tsx` **Lines:** 20?178)
- Import: `useTranslation` hook at top (line 9)
- Extract locale for API: `const locale = i18n.resolvedLanguage ?? i18n.language;` (line 25)
- Props typed as interface (lines 15?18)
- t() on every user-facing string (lines 56, 69, 79, etc.)
- Error handling via `errorText(t, errorState)` (line 52) from `lib/errorText.ts`

**Test structure** (`IngestPanel.test.tsx` **Lines:** 1?101)
``tsx
// @vitest-environment jsdom ? REQUIRED for render tests
import { describe, it, expect, vi, beforeEach, afterEach } from "vitest";
import { render, screen, fireEvent, cleanup } from "@testing-library/react";
import i18n from "../i18n/index"; ? REAL i18n singleton
import { IngestPanel } from "../IngestPanel";

beforeEach(async () => {
  window.localStorage.clear();
  await i18n.changeLanguage("en"); ? guarantees init + known locale
});

afterEach(() => {
  cleanup();
  vi.restoreAllMocks();
});

// Mock fetch:
vi.stubGlobal("fetch", vi.fn(async (url: string | URL, init?: RequestInit) => {
  if (u === "/api/process") return { ok: true, status: 202, json: async () => ({ job_id: "j",
... }) };
  // ...
})));
...

**Test runner:** `vitest` (from `vite.config.ts` **Line:** 16?50)
- Environment: `node` by default; `jsdom` via docblock `// @vitest-environment jsdom`
- Setup file: `web/src/test/setup.ts` (lines 21?22 in vite.config) ? provides localStorage polyfill
- Reporters: text + text-summary
```

- Coverage thresholds: statements 92%, branches 87%, functions 84%, lines 92%

```
**npm scripts** (`package.json` **Lines:** 8?13)
```json
"dev": "vite",
"build": "vite build",
"preview": "vite preview",
"typecheck": "tsc --noEmit",
"test": "vitest run",
"test:coverage": "vitest run --coverage"
```
```

6. ****Persistence for "First-Run Done"****

****Pattern:**** Use `localStorage` like the language switcher does.

```
**i18n example:** `web/src/i18n/index.ts` **Lines:** 31, 60?64
```tsx
export const LANG_STORAGE_KEY = "khelsutra.lang";
// In i18n.init():
detection: {
 order: ["localStorage", "querystring", "navigator"],
 lookupLocalStorage: LANG_STORAGE_KEY,
 caches: ["localStorage"],
}
```
```

```
**For wizard, define:**
```tsx
const WIZARD_COMPLETED_KEY = "khelsutra.wizard.completed";

// In SetupWizard component:
const markCompleted = () => {
 localStorage.setItem(WIZARD_COMPLETED_KEY, "true");
};

// In App.tsx before rendering:
const wizardCompleted = localStorage.getItem(WIZARD_COMPLETED_KEY) === "true";
```
```

****No existing banner/dismissed logic**** ? IngestPanel does not check localStorage; you're the first to add it for the wizard.

7. ****Bundling into the Engine****

****Script:**** `C:\Users\avidu\Projects\bhi-p2b\scripts\bundle\_ui.sh` ****Lines:**** 1?96

****How it works:****

1. Takes khelsutra checkout dir (default `../khelsutra`) or a `.tgz` dist artifact
2. Runs `npm ci && npm run build` to produce `web/dist/` (or reuses existing if `BUILD=0`)
3. Copies `web/dist/\*` ? `backend/ui/` in the engine repo
4. Writes metadata: `ui\_version.json` with `spa\_repo`, `spa\_commit`, `engine\_api\_version`, `content\_hash`, `built\_at`

****Key lines:****

- Line 56?57: `cd "\$WEB" && npm ci && npm run build`
- Line 61: `stage\_dist "\$WEB/dist"`
- Line 80: Extracts `engine\_api\_version` from `backend/main.py`

****Version pinning:**** The script captures `spa\_commit` via `git -C "\$SOURCE" rev-parse HEAD` (line 54), stored in `ui\_version.json`. The engine can check this at startup to warn if

SPA/engine versions are mismatched.

****Implication for wizard:**** The wizard's first step (Gemini key) runs client-side; the engine's GET ``/api/capabilities`` tells the UI what's available. The UI and engine must stay coordinated: if you add new capability fields, bump the ``api_version`` in ``backend/main.py`` and update the ``EngineCapabilities`` type in ``web/src/api/types.ts``.

? Tricky Points / Action Items

1. ****6-language parity is ENFORCED:**** The CI test ``web/src/i18n/hygiene.test.ts:key-parity`` will fail if wizard keys are missing from any locale. You MUST add keys to all 6 catalogs simultaneously with matching keys and placeholders.

2. ****No key-save endpoint exists in engine:**** The Gemini-key step must be ****guided setup**** (show "paste key here / save locally") rather than auto-submission. The engine doesn't have ``POST /api/gemini-key`` ? keys are environment-configured on deployment.

3. ****localStorage must use a unique WIZARD_COMPLETED_KEY:**** Define it at the top of your wizard file, distinct from ``LANG_STORAGE_KEY``, so language changes don't reset wizard state.

4. ****test/setup.ts provides jsdom localStorage polyfill:**** Your tests will work out-of-the-box; no extra setup needed for localStorage mocking.

5. ****Catalog is repo-root, not web/:**** ``/locales`` is at ``C:\Users\avidu\Projects\khelsutra\locales``, hoisted so the marketing site can also consume it (D-CATALOG-HOME pattern). Import paths in ``web/src/i18n/index.ts`` reflect this: ``../../../locales/en/common.json``.

6. ****API client doesn't auto-detect missing /api/capabilities:**** You'll add ``getCapabilities()`` to ``web/src/api/client.ts``, but `App.tsx` won't call it automatically. The wizard component calls it on mount, classifies any errors via ``classifyError()``, and handles missing capability fields gracefully (fallback UI).

7. ****App mount order:**** ``main.tsx`` imports ``i18n`` ****before**** mounting ``<App/>`` (line 4). The wizard is a sibling of `IngestPanel`, so it also auto-gets the current language without extra setup.

****Citations Summary:****

- App mount + state: ``web/src/App.tsx:49?246``
- API client: ``web/src/api/client.ts:1?79, 82?134``
- Types: ``web/src/api/types.ts``
- i18n setup: ``web/src/i18n/index.ts:16?79``
- Key-parity test: ``web/src/i18n/hygiene.test.ts:29?83``
- Component template: ``web/src/components/IngestPanel.tsx:1?178``
- Test template: ``web/src/components/IngestPanel.test.tsx:1?101``
- Test setup: ``web/src/test/setup.ts``
- Package scripts: ``web/package.json:8?13``
- Bundle script: ``C:\Users\avidu\Projects\bhi-p2b\scripts\bundle_ui.sh:1?96``
- Catalogs: ``/locales/{en,kn,hi,es,da,id}/common.json``